

ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC
AN TOÀN VÀ BẢO MẬT THÔNG TIN

ĐỀ TÀI:

MODULAR ENCRYPTION ALGORITHM
FOR IMPROVED SECURE DATA TRANSMISSION

(Theo P. Sri Ram Chandra, G. Venkateswara Rao, G. V. Swamy)

Sinh viên thực hiện:

- 1. Huỳnh Thanh Phong - B2207555**
- 2. Mã Quốc Đạt - B2207514**
- 3. Đinh Hữu Thành - B2207563**

Giảng viên giảng dạy:

GV. Phan Bích Chung

Cần Thơ, ngày **30** tháng **10** năm **2025**

ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC
AN TOÀN VÀ BẢO MẬT THÔNG TIN

ĐỀ TÀI:

MODULAR ENCRYPTION ALGORITHM
FOR IMPROVED SECURE DATA TRANSMISSION

(Theo P. Sri Ram Chandra, G. Venkateswara Rao, G. V. Swamy)

Sinh viên thực hiện:

- 1. Huỳnh Thanh Phong - B2207555**
- 2. Mã Quốc Đạt - B2207514**
- 3. Đinh Hữu Thành - B2207563**

Giảng viên giảng dạy:

GV. Phan Bích Chung

Cần Thơ, ngày **30** tháng **10** năm **2025**

LỜI CẢM ƠN

Nhóm chúng em xin trân trọng gửi lời cảm ơn sâu sắc đến cô Phan Bích Chung, giảng viên phụ trách môn An toàn bảo mật thông tin, người đã nhiệt tình hướng dẫn và đồng hành cùng nhóm trong suốt quá trình thực hiện đề tài. Dưới sự chỉ dẫn của cô qua các buổi trao đổi vào mỗi sáng thứ hai, nhóm có cơ hội thảo luận trực tiếp về các ý tưởng cải tiến thuật toán MEA cũng như định hướng phát triển ứng dụng giấu tin trong ảnh.

Trong quá trình làm việc, cô luôn lắng nghe các trình bày của nhóm, đưa ra những ý kiến phản biện mang tính xây dựng, chỉ rõ các điểm chưa tối ưu trong mô hình mã hóa và các sai sót về mặt logic, đồng thời đề xuất các hướng cải tiến rõ ràng, khả thi cả về mặt kỹ thuật lẫn hình thức trình bày báo cáo khoa học.

Nhờ sự hướng dẫn tận tình, kiến thức chuyên môn sâu rộng và tác phong làm việc nghiêm túc của cô, nhóm đã hiểu sâu sắc hơn về cơ chế hoạt động của thuật toán MEA, đồng thời tự tin hơn trong việc phân tích, đánh giá, và đề xuất các phương án cải tiến thuật toán cũng như ứng dụng thực tiễn.

Chúng em, gồm các thành viên: Huỳnh Thanh Phong, Mã Quốc Đạt, và Đinh Hữu Thành, xin gửi lời tri ân chân thành đến cô vì sự đồng hành và giúp đỡ quý báu trong suốt quá trình thực hiện đề tài này.

Cần Thơ, ngày 30 tháng 10 năm 2025

Nhóm sinh viên thực hiện

Huỳnh Thanh Phong

Đinh Hữu Thành

Mã Quốc Đạt

TÓM TẮT

Trong thời đại công nghệ số phát triển mạnh mẽ, vấn đề bảo mật thông tin và an toàn dữ liệu ngày càng trở nên cấp thiết, đặc biệt trong các hệ thống truyền thông và giao dịch trực tuyến. Mặc dù các thuật toán mã hóa kinh điển như AES, RSA hay DES đã chứng minh được độ tin cậy cao, nhưng trong một số ứng dụng yêu cầu tốc độ xử lý nhanh, cấu trúc linh hoạt và khả năng mở rộng tốt, các thuật toán này vẫn còn tồn tại một số hạn chế nhất định.

Xuất phát từ nhu cầu đó, đề tài “Modular Encryption Algorithm for Improved Secure Data Transmission” được thực hiện nhằm nghiên cứu, cải tiến và triển khai thuật toán Modular Encryption Algorithm (MEA) do Chandra et al., 2018 đề xuất. Trong quá trình thực hiện đề tài, nhóm đã xem xét, đánh giá và đưa ra bốn hướng cải tiến mới, nhằm nâng cao hiệu quả bảo mật và khả năng ứng dụng trong thực tế:

1. **Thuật toán biến đổi dữ liệu trên phép đơn ánh phi tuyến bậc 2** $\mathbb{R}^n \rightarrow \mathbb{R}^m$: giúp mở rộng khả năng ánh xạ dữ liệu từ không gian đầu vào sang không gian có chiều khác, tăng độ phức tạp và khó dự đoán trong quá trình mã hóa.
2. **Thuật toán hoán vị trên nhóm tuyến tính tổng quát bậc 3 trên trường hữu hạn** F_n : ứng dụng lý thuyết đại số trừu tượng nhằm tạo ra cơ chế hoán vị phi tuyến mạnh hơn, tăng khả năng chống lại các tấn công phân tích mật mã tuyến tính.
3. **Thuật toán giấu ảnh trong kênh màu RGBA**: kết hợp cơ chế mã hóa và kỹ thuật giấu thông tin (steganography) để bảo vệ dữ liệu trực quan, phù hợp cho ứng dụng truyền hình ảnh và đa phương tiện an toàn.
4. **Dùng thuật toán DES mã hóa giá trị phân mảnh**: Xuất phát từ ý tưởng hàm băm, nội dung được chia làm nhiều khối lưu vào ảnh và được xác định bằng 64 bits phân mảnh và được mã hóa bằng DES.

Trong đề án, nhóm tiến hành phân tích lý thuyết, thiết kế mô hình mã hóa – giải mã, cài đặt thử nghiệm bằng Python và đánh giá hiệu năng so sánh với các thuật toán truyền thống. Tiến hành thử nghiệm trên máy RAM 8GB, SSD 512GB, Intel Core i5-1135G7 2.40GHz kết quả đạt hiệu quả cao. Thời gian sinh khóa tăng từ 1.34s lên 1.405s, thời gian mã hóa tăng từ 0.160s lên 0.350s và thời gian giải mã tăng từ 1.076s lên 1.458s. Sự đánh đổi thời gian mang lại độ an toàn cao với $MSE = 0$, $PSNR = 148.131$ dB, Entropy trước và sau đạt xấp xỉ 7.369 bits. Cũng với giá trị lũy tiến trên 200 bits thay đổi plain text là 0.3816 cho thấy MEA tăng cường nhạy với các tấn công sai biệt, tấn công thống kê.

ABSTRACT

In the era of strong digital technology development, the issue of information security and data safety is becoming increasingly urgent, especially in communication systems and online transactions. Although classic encryption algorithms such as AES, RSA or DES have proven to be highly reliable, in some applications that require fast processing speed, flexible structure and good scalability, these algorithms still have certain limitations.

Based on that need, the topic “Modular Encryption Algorithm for Improved Secure Data Transmission” was carried out to research, improve and deploy Modular Encryption Algorithm (MEA) proposed by Chandra et al., 2018. During implementation of the project, the group reviewed, evaluated and proposed four new directions of improvement, aiming to improve security efficiency and practical applicability:

1. **Data transformation algorithm on nonlinear injection of degree 2** $\mathbb{R}^n \rightarrow \mathbb{R}^m$: helps to expand the ability to map data from the input space to a space of other dimensions, increasing complexity and unpredictability in the encryption process.
2. **Permutation algorithm on general linear group of degree 3 on finite field F_n** : applies abstract algebraic theory to create a stronger nonlinear permutation mechanism, increasing the ability to resist linear cryptanalysis attacks.
3. **Image hiding algorithm in RGBA color channel**: combines encryption mechanism and information hiding technique (steganography) to protect visual data, suitable for secure image transmission and multimedia applications.
4. **Using DES algorithm to encrypt fragmented values**: Based on the idea of hash function, the content is divided into many blocks stored in the image and identified by 64 fragment bits and encrypted with DES.

In the project, the team conducted theoretical analysis, design of encryption-decryption model, experimental installation using Python, and performance evaluation compared with traditional algorithms. Testing on a machine with 8GB RAM, 512GB SSD, Intel Core i5-1135G7 2.40GHz resulted in high efficiency. Key generation time increased from 1.34s to 1.405s, encryption time increased from 0.160s to 0.350s and decryption time increased from 1.076s to 1.458s. The time trade-off brings high security with MSE = 0, PSNR = 148.131 dB, Entropy before and after reaching approximately 7.369 bits. Also with the progressive value over 200 bits of plain text change is 0.3816 showing that MEA increases sensitivity to differential attacks, statistical attacks.

MỤC LỤC

DANH MỤC HÌNH	1
DANH MỤC BẢNG	2
1 CƠ SỞ LÝ THUYẾT BÀI BÁO	3
1.1 MÃ HÓA ĐỐI XỨNG	3
1.2 SƠ LƯỢC NỘI DUNG CỦA TÁC GIẢ	4
1.2.1 Giới thiệu M-Box – thành phần mới của MEA	4
1.2.2 Lý thuyết về ma trận tam thức	4
1.2.3 Thiết kế và phân tích các bảng Log, Exp và S-Box	5
1.2.4 Phân tích hiệu năng và độ phức tạp	5
1.2.5 Đánh giá tính an toàn	5
1.3 CƠ SỞ GIẢI THUẬT MEA - Modular Encryption Algorithm	6
1.3.1 Giải thuật mã hóa	6
1.3.2 Giải thuật giải mã	9
1.3.3 Giải thuật MEA M-box	11
2 CƠ SỞ LÝ THUYẾT CẢI TIẾN	13
2.1 SƠ LƯỢC NỘI DUNG PHẦN NHÓM CẢI TIẾN	13
2.2 CƠ SỞ TRƯỜNG $GL_3(\mathbb{F}_n)$ BẰNG PHÉP LIÊN HỢP VÀ LŨY THỪA RỜI RẠC TRÊN TRƯỜNG HỮU HẠN - HOÁN VỊ ĐẦU	14
2.2.1 Mã hóa ma trận dựa trên phép liên hợp và lũy thừa rời rạc	14
2.2.2 Quy trình mã hóa – giải mã	15
2.2.3 Ý nghĩa toán học	16
2.3 CƠ SỞ HÀM PHI TUYẾN BẬC 2 - HOÁN VỊ PHI TUYẾN BẬC 2	17
2.3.1 Hàm mã hóa	17
2.3.2 Hàm giải mã	18
2.4 CƠ SỞ GIẤU ẢNH TRONG KÊNH MÀU RGBA	19
2.4.1 Biểu diễn ảnh số	19
2.4.2 Chuyển đổi dữ liệu giấu	19
2.4.3 Nguyên lý nhúng dữ liệu	20
2.4.4 Giải mã dữ liệu	21
2.4.5 Mô hình nhúng và trích xuất	21
2.4.6 Độ sai khác và đánh giá chất lượng	22
2.5 CƠ SỞ THUẬT TOÁN DES - Data Encryption Standard	22
2.6 Kết luận	24
3 QUY TRÌNH THỰC HIỆN THUẬT TOÁN MEA (Modular Encryption Algorithm) CẢI TIẾN	25
3.1 Sinh Khóa	26
3.2 Thuật toán mã hóa	27
3.3 Thuật toán giải mã	30

4	KẾT QUẢ THỰC NGHIỆM	34
5	TỔNG KẾT VÀ ĐỊNH HƯỚNG PHÁT TRIỂN	38
5.1	BÀI HỌC TIẾP THU	38
5.2	ƯU ĐIỂM	38
5.3	HẠN CHẾ	39
5.4	HƯỚNG PHÁT TRIỂN	39

DANH MỤC HÌNH

1.1	Sơ đồ mã hóa đối xứng	3
1.2	Sơ đồ giải thuật Modular Encryption Algorithm	6
1.3	Giá trị của bảng Logarithms	7
1.4	Giá trị của bảng Exponentials.	8
1.5	Giá trị của bảng Substitution Box.	9
1.6	Giá trị của bảng Inverse Sub-Bytes.	10
2.1	Tổ chức cấu trúc dấu trong ảnh	20
2.2	Cấu trúc ảnh RGBA và kênh Alpha được sử dụng để giấu dữ liệu.	21
2.3	Cấu trúc của giải thuật DES	23
3.1	Sơ đồ giải thuật Modular Encryption Algorithm cải tiến phần 1	25
3.2	Sơ đồ giải thuật Modular Encryption Algorithm cải tiến phần 2	26
4.1	Đánh giá mô hình MEA	35
4.2	So sánh giữa ảnh gốc và ảnh sau khi giấu tin	35
4.3	So sánh histogram màu ảnh trước và sau khi dấu tin	36

DANH MỤC BẢNG

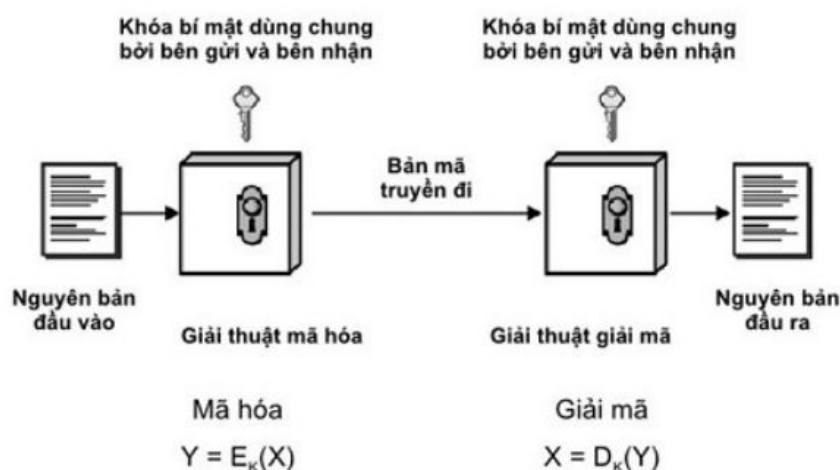
2.1	Giá trị các lũy thừa $3^k \bmod 17$ trong $\mathbb{F}_{17}^\times$	15
2.2	Bảng giá trị $\log_3(a) \bmod 16$ trong $\mathbb{F}_{17}^\times$	15
2.3	Mô tả các giá trị hợp lệ của B theo p	17
2.4	Quy trình giải mã các giá trị x_i từ y theo cơ số B và hằng số p	19
3.1	Cấu trúc khóa trong 5 bước sinh dữ liệu	27
3.2	Quy trình giải mã các giá trị t_{ij} từ y theo cơ số B và hằng số p	31
4.1	Cấu hình máy tính và môi trường thực nghiệm	34
4.2	So sánh thời gian thực thi giữa MEA và MEA cải tiến	35
4.3	So sánh chất lượng ảnh giữa ảnh gốc và ảnh sau khi giấu tin	36
4.4	Minh họa Avalanche Effect với các số bit thay đổi khác nhau	37

Chương 1

CƠ SỞ LÝ THUYẾT BÀI BÁO

1.1 MÃ HÓA ĐỐI XỨNG

Mô hình hệ mã hóa đối xứng



Hình 1.1: Sơ đồ mã hóa đối xứng

Mật mã đối xứng (Symmetric Encryption) là loại mã hóa mà cùng một khóa bí mật được dùng cho cả hai quá trình:

- Mã hóa (Encryption): chuyển bản rõ (plaintext) thành bản mã (ciphertext);
- Giải mã (Decryption): chuyển bản mã trở lại bản rõ.

Điều này có nghĩa là người gửi và người nhận phải chia sẻ cùng một khóa và giữ bí mật tuyệt đối về khóa đó.

Ví dụ: AES, DES, Blowfish, và MEA (Modular Encryption Algorithm) đều là thuật toán mã hóa đối xứng.

1.2 SƠ LƯỢC NỘI DUNG CỦA TÁC GIẢ

Theo bài báo gốc “*Modular Encryption Algorithm for Secure Data Transmission*” của P. Sri Ram Chandra, G. Venkateswara Rao, và G. V. Swamy (2018), nhóm tác giả đã:

- Đề xuất một thuật toán mã hóa khối đối xứng mới tên là Modular Encryption Algorithm (MEA).
- MEA dùng ma trận tri-modular (determinant = 3) để sinh khóa.
- Quá trình mã hóa thực hiện 19 vòng: 16 vòng ở pha I và 3 vòng ở pha II.
- Sử dụng các phép toán hoán vị (Permutation) và thay thế (Substitution), tương tự như trong AES.

1.2.1 Giới thiệu M-Box – thành phần mới của MEA

- M-Box là hộp nén dữ liệu (*Compression Box*), chuyển 4 ký tự hexa thành 2 ký tự hexa.
- Nó đảm bảo tính khuếch tán (diffusion) và nhiễu loạn (confusion) theo nguyên lý của Shannon.
- Giúp MEA chống lại tấn công sai biệt (Differential Cryptanalysis).

1.2.2 Lý thuyết về ma trận tam thức

Định nghĩa

Một ma trận tam thức 3×3 là ma trận

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix},$$

và nó liên quan đến tam thức bậc hai trong ba biến x_1, x_2, x_3 :

$$Q(x_1, x_2, x_3) = a_{11}x_1^2 + a_{22}x_2^2 + a_{33}x_3^2 + (a_{12} + a_{21})x_1x_2 + (a_{13} + a_{31})x_1x_3 + (a_{23} + a_{32})x_2x_3.$$

Nếu ma trận đối xứng ($a_{ij} = a_{ji}$), biểu thức sẽ đơn giản hơn:

$$Q(x_1, x_2, x_3) = a_{11}x_1^2 + a_{22}x_2^2 + a_{33}x_3^2 + 2a_{12}x_1x_2 + 2a_{13}x_1x_3 + 2a_{23}x_2x_3.$$

Các tính chất quan trọng

1. **Đối xứng:** Ma trận tam thức 3×3 thường được giả sử là đối xứng.
2. **Định thức và hạng:**
 - $\det(A) \neq 0 \Rightarrow$ tam thức *không suy biến* (non-degenerate).
 - Hạng $\text{rank}(A)$ bằng số biến độc lập trong tam thức.
3. **Biến đổi tuyến tính:** Nếu $y = Px$, thì tam thức trở thành

$$Q(y) = y^T(P^TAP)y.$$

4. **Chuẩn hóa:** Tam thức có thể được đưa về dạng chuẩn (chỉ có các bình phương) bằng phương pháp *chuyển vị chính tắc*.

1.2.3 Thiết kế và phân tích các bảng Log, Exp và S-Box

- Các bảng này giúp khuếch tán dữ liệu tốt hơn.
- Đảm bảo tiêu chí Strict Plaintext Avalanche Criterion (SPAC) và Strict Key Avalanche Criterion (SKAC) $> 99\%$, nghĩa là thay đổi nhỏ trong bản rõ hoặc khóa dẫn đến thay đổi lớn trong bản mã.

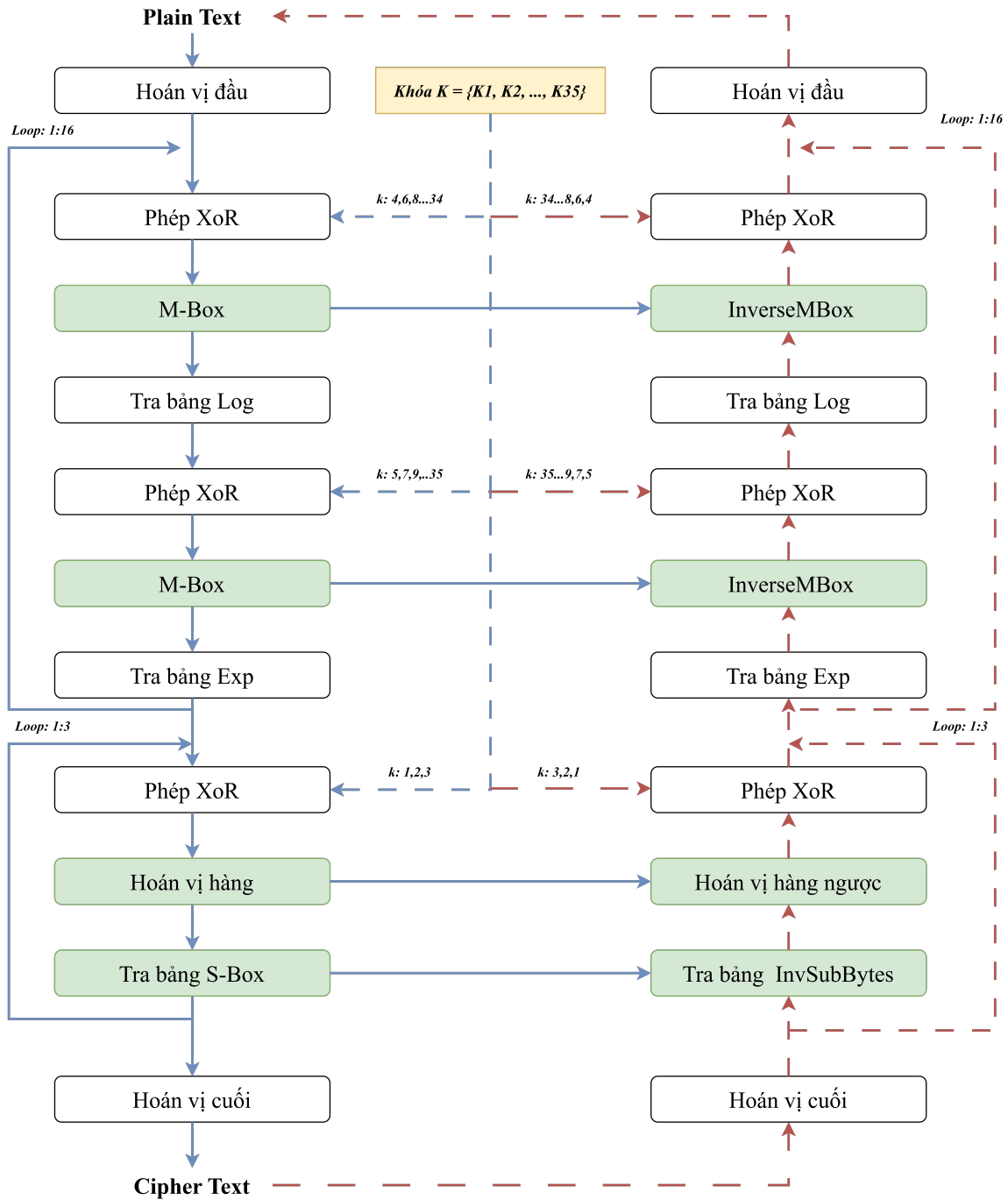
1.2.4 Phân tích hiệu năng và độ phức tạp

- Mã hóa khối 72-bit (9 ký tự ASCII).
- Sử dụng 35 ma trận tri-modular (5040-bit key tổng hợp).
- Thời gian sinh khóa và mã hóa/giải mã nhanh, độ phức tạp $O(n^2)$.

1.2.5 Đánh giá tính an toàn

- Khó bị phá bằng brute-force hoặc tấn công phân tích khóa.
- Thỏa mãn đầy đủ tiêu chí về tính bảo mật của hệ mật đối xứng.

1.3 CƠ SỞ GIẢI THUẬT MEA - Modular Encryption Algorithm



Hình 1.2: Sơ đồ giải thuật Modular Encryption Algorithm

1.3.1 Giải thuật mã hóa

Đầu vào (Input): Chuỗi bản rõ (*Plaintext*) được chia thành các khối gồm 9 ký tự. Mỗi khối 9 ký tự tương ứng với $9 \times 8 = 72$ bit dữ liệu.

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix}$$

trong đó a_{00} đến a_{22} là các giá trị ASCII của 9 ký tự tương ứng, được biểu diễn ở dạng Hexadecimal (cơ số 16).

Bước 1: Hoán vị khởi tạo (Initial Permutation)

Áp dụng phép hoán vị ban đầu cho ma trận A để thu được ma trận $A^{(2)}$ như sau:

$$A^{(2)} = \begin{bmatrix} a_{21} & a_{22} & a_{01} \\ a_{02} & a_{00} & a_{10} \\ a_{11} & a_{20} & a_{12} \end{bmatrix}$$

Pha I (Phase-I): Vòng lặp 16 lần

For loop: $i = 1$ đến 16 thực hiện các bước sau:

1. **Phép XOR với khóa:**

$$A^{(3)} = A^{(2)} \oplus K_i$$

trong đó K_i là ma trận khóa tam thức (*Tri-Modular matrix*) với $4 \leq i \leq 34$.

2. **Phép nén (Compression Permutation)** qua hộp M-Box:

$$A^{(4)} = M\text{-Box}(A^{(3)})$$

3. **Bảng logarit (Log Table):**

$$A^{(5)} = \log(A^{(4)})$$

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		00	19	01	32	02	1A	C6	4B	C7	1B	68	33	EE	DF	03
1	64	04	E0	0E	34	8D	81	EF	4C	71	08	C8	F8	69	1C	C1
2	7D	C2	1D	B5	F9	B9	27	6A	4D	E4	A6	72	9A	C9	09	78
3	65	2F	8A	05	21	0F	E1	24	12	F0	82	45	35	93	DA	8E
4	96	8F	DB	BD	36	D0	CE	94	13	5C	D2	F1	40	46	83	38
5	66	DD	FD	30	BF	06	8B	62	B3	25	E2	98	22	88	91	10
6	7E	6E	48	C3	A3	B6	1E	42	3A	6B	28	54	FA	85	3D	BA
7	2B	79	0A	15	9B	9F	5E	CA	4E	D4	AC	E5	F3	73	A7	57
8	AF	58	A8	50	F4	EA	D6	74	4F	AE	E9	D5	E7	E6	AD	E8
9	2C	D7	75	7A	EB	16	0B	F5	59	CB	5F	B0	9C	A9	51	A0
A	7F	0C	F6	6F	17	C4	49	EC	D8	43	1F	2D	A4	76	7B	B7
B	CC	BB	3E	5A	FB	60	B1	86	3B	52	A1	6C	AA	55	29	9D
C	97	B2	87	90	61	BE	DC	FC	BC	95	CF	CD	37	3F	5B	D1
D	53	39	84	3C	41	A2	6D	47	14	2A	9E	5D	56	F2	D3	AB
E	44	11	92	D9	23	20	2E	89	B4	7C	B8	26	77	99	E3	A5
F	67	4A	ED	DE	C5	31	FE	18	OD	63	8C	80	C0	F7	70	07

Hình 1.3: Giá trị của bảng Logarithms .

4. **Phép XOR với khóa mới:**

$$A^{(6)} = A^{(5)} \oplus K_i$$

với $5 \leq i \leq 35$.

5. Phép nén lần 2:

$$A^{(7)} = M\text{-Box}(A^{(6)})$$

6. Bảng mũ (Exponential Table):

$$A^{(8)} = \exp(A^{(7)})$$

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	01	03	05	0F	11	33	55	FF	1A	2E	72	96	A1	F8	13	35
1	5F	E1	38	48	D8	73	95	A4	F7	02	06	0A	1E	22	66	AA
2	E5	34	5C	E4	37	59	EB	26	6A	BE	D9	70	90	AB	E6	31
3	53	F5	04	0C	14	3C	44	CC	4F	D1	68	B8	D3	6E	B2	CD
4	4C	D4	67	A9	E0	3B	4D	D7	62	A6	F1	08	18	28	78	88
5	83	9E	B9	D0	6B	BD	DC	7F	81	98	B3	CE	49	DB	76	9A
6	B5	C4	57	F9	10	30	50	F0	0B	1D	27	69	BB	D6	61	A3
7	FE	19	2B	7D	87	92	AD	EC	2F	71	93	AE	E9	20	60	A0
8	FB	16	3A	4E	D2	6D	B7	C2	5D	E7	32	56	FA	15	3F	41
9	C3	5E	E2	3D	47	C9	40	C0	5B	ED	2C	74	9C	BF	DA	75
A	9F	BA	D5	64	AC	EF	2A	7E	82	9D	BC	DF	7A	8E	89	80
B	9B	B6	C1	58	E8	23	65	AF	EA	25	6F	B1	C8	43	C5	54
C	FC	1F	21	63	A5	F4	07	09	1B	2D	77	99	B0	CB	46	CA
D	45	CF	4A	DE	79	8B	86	91	A8	E3	3E	42	C6	51	F3	0E
E	12	36	5A	EE	29	7B	8D	8C	8F	8A	85	94	A7	F2	0D	17
F	39	4B	DD	7C	84	97	A2	FD	1C	24	6C	B4	C7	52	F6	01

Hình 1.4: Giá trị của bảng Exponentials.

Pha II (Phase-II): Vòng lặp 3 lần

For loop: $i = 1$ đến 3 thực hiện các bước sau:

1. Phép XOR với khóa tam thức:

$$A^{(9)} = A^{(8)} \oplus K_i, \quad 1 \leq k \leq 3$$

2. Hoán vị hàng (Row Interchange):

$$R_i \leftarrow R_{(i+1) \bmod 3}, \quad 0 \leq i \leq 2$$

tức là hàng thứ i được thay thế bằng hàng kế tiếp (theo chu kỳ modulo 3).

3. Phép thay thế byte (Sub-Bytes):

$$A^{(11)} = S\text{-Box}(A^{(10)})$$

Bước 12: Chuyển đổi sang giá trị thập phân

$$A^{(12)} = \text{Decimal}(A^{(11)})$$

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Hình 1.5: Giá trị của bảng Substitution Box.

Bước 13: Chuyển về ký tự ASCII

$$A^{(13)} = \text{ASCII}(A^{(12)})$$

Tức là:

$$A^{(13)} = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} \Leftarrow \begin{bmatrix} a_{21} & a_{22} & a_{01} \\ a_{02} & a_{00} & a_{10} \\ a_{11} & a_{20} & a_{12} \end{bmatrix}$$

Bước 14: Đầu ra (Output)

Khối bản mã (*Cipher-text block*) cuối cùng gồm 9 ký tự, tương ứng 72 bit:

$$\text{Ciphertext} = A^{(13)}$$

1.3.2 Giải thuật giải mã

Đầu vào (Input): Chuỗi bản mã (*Cipher-text*) được chia thành các khối gồm 9 ký tự. Mỗi khối 9 ký tự tương ứng với $9 \times 8 = 72$ bit dữ liệu.

$$M^{(1)} = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix}$$

trong đó a_{00} đến a_{22} là các giá trị ASCII thập phân của 9 ký tự Cipher-text đầu vào.

Bước 1: Hoán vị khởi tạo (Initial Permutation)

Áp dụng hoán vị ban đầu trên ma trận thập phân:

$$M^{(2)} = \begin{bmatrix} a_{21} & a_{22} & a_{01} \\ a_{02} & a_{00} & a_{10} \\ a_{11} & a_{20} & a_{12} \end{bmatrix} \Leftarrow \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix}$$

Pha I (Phase-I): Vòng lặp 3 lần

For loop: $i = 1$ đến 3 thực hiện các bước sau:

1. Chuyển đổi thập phân sang hệ 16 (Decimal $\rightarrow$ Hex):

$$M^{(3)} = \text{Dec2Hex}(M^{(2)})$$

2. Phép *Inverse SubBytes*:

$$M^{(4)} = \text{InvSubBytes}(M^{(3)})$$

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
1	7C	E3	39	82	9B	2F	FF	87	34	83	43	81	C4	DE	E9	CB
2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	C4	42	FA	C3	4E
3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
5	6C	70	A8	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

Hình 1.6: Giá trị của bảng Inverse Sub-Bytes.

3. Hoán vị hàng (Row Interchange): với quy tắc tuần hoàn

$$R_{(i+1) \bmod 3} \leftarrow R_i, \quad 0 \leq i \leq 2$$

thu được $M^{(5)}$.

4. XOR với khóa tam thức:

$$M^{(6)} = M^{(5)} \oplus K_i$$

trong đó K_i là các ma trận khóa tam thức ứng với $1 \leq k \leq 3$.

Pha II (Phase-II): Vòng lặp 16 lần

For loop: $j = 1$ đến 16 thực hiện các bước sau:

1. Tra bảng logarithm:

$$M^{(7)} = \log(M^{(6)})$$

2. Phép nén/khôi phục mở rộng (Expansion permutation) qua *Inverse M-Box*:

$$M^{(8)} = \text{InverseMBox_Expand}(M^{(7)})$$

3. XOR với khóa tam thức:

$$M^{(9)} = M^{(8)} \oplus K_j \quad (\text{với } 5 \leq k \leq 35 \text{ theo quy định khóa})$$

4. Tra bảng Exponential:

$$M^{(10)} = \exp(M^{(9)})$$

5. Lại phép mở rộng qua *Inverse M-Box*:

$$M^{(11)} = \text{InverseMBox_Expand}(M^{(10)})$$

6. XOR với khóa tam thức:

$$M^{(12)} = M^{(11)} \oplus K_j \quad (\text{với } 4 \leq k \leq 34 \text{ theo quy định khóa})$$

7. Gán $M^{(6)} \leftarrow M^{(12)}$ cho vòng kế tiếp (nếu có).

Bước 12: Chuyển từ hệ Hex về hệ thập phân

$$M^{(13)} = \text{Hex2Dec}(M^{(12)})$$

Bước 13: Chuyển sang ký tự ASCII

$$M^{(14)} = \text{ASCII}(M^{(13)})$$

trong đó ma trận kết quả được sắp về dạng ban đầu:

$$M^{(14)} = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} \Leftarrow \begin{bmatrix} a_{21} & a_{22} & a_{01} \\ a_{02} & a_{00} & a_{10} \\ a_{11} & a_{20} & a_{12} \end{bmatrix}$$

Bước 14: Đầu ra (Output)

Khối bản rõ (Plaintext block) cuối cùng gồm 9 ký tự, tương ứng 72 bit:

$$\text{Plaintext} = M^{(14)}$$

1.3.3 Giải thuật MEA M-box

Đầu vào (Input): Một khối gồm 4 chữ số Hexadecimal (ví dụ: 2ACF).

Bước 1: Biểu diễn dưới dạng nhị phân

Mỗi chữ số Hexadecimal được biểu diễn bằng 4 bit nhị phân. Do đó, 4 ký tự Hex tương ứng với:

$$4 \times 4 = 16 \text{ bit}$$

Ví dụ:

$$2A = 0010 \ 1010, \quad CF = 1100 \ 1111$$

Bước 2: Chia dãy bit thành hai phần và biểu diễn dưới dạng đa thức

Chia dãy 16 bit thành 2 phần bằng nhau, mỗi phần gồm 8 bit. Mỗi phần được biểu diễn thành một đa thức nhị phân theo bậc của x .

Ví dụ:

$$\begin{aligned} 2A &\Rightarrow x^5 + x^3 + x^1, \\ CF &\Rightarrow x^7 + x^6 + x^3 + x^2 + x^1 + x^0. \end{aligned}$$

Bước 3: Nhân hai đa thức

Thực hiện phép nhân hai đa thức vừa thu được bằng phép nhân đa thức thông thường (không rút gọn theo modulo ở bước này):

$$P(x) = P_1(x) \times P_2(x)$$

Kết quả là một đa thức bậc cao (thường lớn hơn 8).

Bước 4: Chọn các hạng tử có hệ số lẻ và chia modulo $11B_{16}$

Từ đa thức kết quả $P(x)$, chỉ giữ lại các hạng tử có **hệ số lẻ** (bỏ các hệ số chẵn). Bỏ hệ số, chỉ giữ phần lũy thừa của x .

Chương 2

CƠ SỞ LÝ THUYẾT CẢI TIẾN

2.1 SƠ LƯỢC NỘI DUNG PHẦN NHÓM CẢI TIẾN

Trong nghiên cứu này, nhóm em kế thừa cấu trúc lõi của MEA gốc và đề xuất ba cải tiến chính nhằm tăng cường bảo mật và mở rộng ứng dụng thực tế trong giấu tin ảnh (image steganography):

(1) Cải tiến hoán vị đầu (Initial Permutation cải tiến)

Thay vì hoán đổi vị trí cố định như MEA gốc, nhóm em dùng ánh xạ tuyến tính trên trường hữu hạn:

$$X' = S \cdot [g^{x_{ij} \bmod(p)}] \cdot S^{-1} \quad (S \in GL_3(\mathbb{F}_p), g \in \mathbb{F}_p^*)$$

Tạo tính ngẫu nhiên cao, chống phân tích tuyến tính, và đảm bảo ánh xạ đơn ánh.

(2) Bổ sung hoán vị phi tuyến bậc hai (Quadratic Nonlinear Permutation)

Sau khi MEA sinh bản mã, áp dụng hàm phi tuyến:

$$g(x) = x + px^2 \Rightarrow y = \sum g(x_i)B^i + p \sum x_i^2 B^i$$

với p là số nguyên tố lớn, B là cơ số, và $y \leq y_{max}$.

⇒ Tăng tính hỗn loạn, cải thiện diffusion, và chống suy luận tuyến tính.

(3) Tích hợp giấu tin trong ảnh RGBA bằng LSB kết hợp DES

- Chia bản mã thành nhiều phần.
- 64 bit đầu của mỗi ảnh được mã hóa bằng **DES (64-bit key)** để lưu header (số ảnh, thứ tự).
- Phần dữ liệu còn lại được nhúng vào kênh Alpha (A) theo quy tắc:

$$A' = \begin{cases} 253, & \text{nếu bit không chứa thông tin} \\ 254, & \text{nếu bit} = 0 \\ 255, & \text{nếu bit} = 1 \end{cases}$$

- Kết quả:

- PSNR > 50 dB, ảnh sau giấu không thay đổi trực quan.
- Dung lượng giấu cao ($\sim 99\%$ pixel).

2.2 CƠ SỞ TRƯỜNG $GL_3(\mathbb{F}_n)$ BẰNG PHÉP LIÊN HỢP VÀ LŨY THỪA RỜI RẠC TRÊN TRƯỜNG HỮU HẠN - HOÁN VỊ ĐẦU

2.2.1 Mã hóa ma trận dựa trên phép liên hợp và lũy thừa rời rạc

Giả sử p là một số nguyên tố lớn, g là phần tử sinh của nhóm nhân $\mathbb{F}_p^\times$, và $S \in GL(3, \mathbb{F}_p)$ là ma trận khả nghịch (được chọn làm khóa bí mật).

Để một ma trận S thuộc nhóm $GL(3, \mathbb{F}_p)$, ta phải có điều kiện:

$$\det(S) \not\equiv 0 \pmod{p}.$$

Hay nói cách khác, định thức của ma trận S không được chia hết cho p . Khi đó tồn tại ma trận nghịch đảo S^{-1} thỏa mãn:

$$S \cdot S^{-1} \equiv I_3 \pmod{p},$$

trong đó I_3 là ma trận đơn vị bậc 3.

Cho ma trận thông điệp

$$X = (x_{ij}), \quad x_{ij} \in [0, 15] \subset \mathbb{Z}_{p-1}.$$

Trong đó, mỗi phần tử x_{ij} có thể được biểu diễn dưới dạng **giá trị thập lục phân (hexadecimal)**:

$$x_{ij} \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}.$$

a. Hàm mã hóa

Hàm mã hóa được định nghĩa như sau:

$$f(X) = S [g^{x_{ij} \bmod(p)}] S^{-1}.$$

Trong đó:

- Mỗi phần tử x_{ij} được ánh xạ thành $g^{x_{ij}}$ trên trường $\mathbb{F}_p$;
- Phép nhân $S[g^{x_{ij} \bmod(p)}]S^{-1}$ là phép liên hợp ma trận (matrix conjugation).

Hàm f biến đổi ma trận X thành ma trận $Y = f(X)$, đóng vai trò là bản mã (ciphertext).

b. Hàm giải mã

Với khóa bí mật S , hàm giải mã được định nghĩa:

$$g(Y) = \left(\log_g \left((S^{-1}YS)_{ij} \right) \bmod (p-1) \right)_{ij}.$$

Ý tưởng:

1. Tính ma trận $S^{-1}YS = [g^{x_{ij} \bmod(p)}]$;
2. Lấy $\log_g$ từng phần tử để thu lại x_{ij} .

Khi đó, ta có $g(f(X)) = X$.

2.2.2 Quy trình mã hóa – giải mã

Bước	Mã hóa	Giải mã
1	Cho ma trận $X = (x_{ij})$	Nhận bản mã Y
2	Tính $M = [g^{x_{ij}} \bmod (p)]$	Tính $M' = S^{-1}YS$
3	Tính $Y = SM S^{-1}$	Tính $X = [\log_g(M'_{ij})] \bmod (p-1)$

Bảng 2.1: Giá trị các lũy thừa $3^k \bmod 17$ trong $\mathbb{F}_{17}^\times$

k	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$3^k \bmod 17$	1	3	9	10	13	5	15	11	16	14	8	7	4	12	2	6

Bảng 2.2: Bảng giá trị $\log_3(a) \bmod 16$ trong $\mathbb{F}_{17}^\times$

a	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\log_3(a) \bmod 16$	0	14	1	12	5	15	11	10	2	3	7	13	4	9	6	8

Ví dụ minh họa

Thiết lập:

- Chọn số nguyên tố $p = 17$.
- Chọn phần tử sinh $g = 3$ của nhóm nhân $\mathbb{F}_{17}^\times$.
- Lấy ma trận khóa $S \in GL(3, \mathbb{F}_{17})$ và ma trận thông điệp X có các phần tử thuộc $\mathbb{Z}_{p-1} = \mathbb{Z}_{16}$.

Hàm mã hoá và giải mã tổng quát:

$$f(X) = S[g^{x_{ij}} \bmod (p)]S^{-1}, \quad g(Y) = \left(\log_g \left((S^{-1}YS)_{ij} \right) \bmod (p-1) \right)_{ij}.$$

Dữ liệu ví dụ:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad S = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}.$$

Bước 1. Tạo ma trận $M = [g^{x_{ij}}]$ trong $\mathbb{F}_{17}$.

Tính $g^{x_{ij}} \bmod 17$ với $g = 3$. Ta được

$$M = \begin{bmatrix} 3^1 & 3^2 & 3^3 \\ 3^4 & 3^5 & 3^6 \\ 3^7 & 3^8 & 3^9 \end{bmatrix} \bmod 17 = \begin{bmatrix} 3 & 9 & 10 \\ 13 & 5 & 15 \\ 11 & 16 & 14 \end{bmatrix}.$$

Bước 2. Tính $S^{-1} \pmod{17}$.

Tính nghịch đảo ma trận S theo modulo 17, ta có

$$S^{-1} \equiv \begin{bmatrix} 10 & 1 & 5 \\ 3 & 2 & 13 \\ 12 & 4 & 1 \end{bmatrix} \pmod{17}.$$

Bước 3. Mã hoá: $Y = S M S^{-1}$ (tất cả modulo 17).

Thực hiện nhân ma trận theo modulo 17, ta được

$$Y \equiv S M S^{-1} \pmod{17} = \begin{bmatrix} 3 & 14 & 5 \\ 14 & 3 & 12 \\ 13 & 4 & 16 \end{bmatrix}.$$

Bước 4. Giải mã:

1. Tính $M' = S^{-1} Y S \pmod{17}$. Vì $Y = S M S^{-1}$, ta sẽ thu lại $M' = M$. Thực tế tính được:

$$M' \equiv \begin{bmatrix} 3 & 9 & 10 \\ 13 & 5 & 15 \\ 11 & 16 & 14 \end{bmatrix} \pmod{17}.$$

2. Với mỗi phần tử m'_{ij} trong M' , ta lấy logarit rời rạc cơ sở $g = 3$ trong $\mathbb{F}_{17}^\times$. Từ đó xác định logarit rời rạc từng phần tử:

$$\log_3 \begin{pmatrix} 3 & 9 & 10 \\ 13 & 5 & 15 \\ 11 & 16 & 14 \end{pmatrix} \pmod{16} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}.$$

Kết quả giải mã đúng bằng ma trận thông điệp ban đầu X .

Ghi chú về bảo mật và thực hành:

- Khóa S phải được chọn sao cho S khả nghịch modulo p (tức $\det(S) \not\equiv 0 \pmod{p}$).
- Việc tính logarit rời rạc (discrete logarithm) là phần bắt buộc cho giải mã trong ví dụ này — trong thực tế, bài toán log rời rạc khó khăn định độ an toàn cho một số mô hình thuật toán mật mã.
- Ở ví dụ minh họa ta dùng số nhỏ cho dễ tính; môi trường thực tế phải dùng p lớn để đảm bảo an toàn.

2.2.3 Ý nghĩa toán học

Phương pháp này kết hợp hai nguyên lý chính:

- **Mật mã lũy thừa rời rạc (Discrete Logarithm Cryptography):** dựa trên bài toán khó $\log_g(a)$ trong $\mathbb{F}_p^\times$.

- **Phép liên hợp ma trận (Matrix Conjugation):** giúp ẩn cấu trúc nội tại của dữ liệu và tăng độ phức tạp khi tấn công.

Do đó, hệ mã hóa này có thể được ứng dụng trong:

- Mã hóa dữ liệu dạng ma trận (ảnh, tín hiệu, văn bản);
- Xây dựng hệ mật mã khối dựa trên nhóm tuyến tính tổng quát $GL(n, \mathbb{F}_p)$.

2.3 CƠ SỞ HÀM PHI TUYẾN BẬC 2 - HOÁN VỊ PHI TUYẾN BẬC 2

2.3.1 Hàm mã hóa

Chọn hằng số p nguyên tùy ý trên trường số nguyên và cơ số $B > p \cdot 15^2 + 15$ sao cho $y \leq y_{\max}$.

Đặt:

$$g(x) = x + px^2.$$

Khi đó:

$$y = g(x_1) + B g(x_2) + B^2 g(x_3) + B^3 g(x_4)$$

Cụ thể hơn, thay $g(x_i) = x_i + px_i^2$ ta được:

$$\begin{aligned} y &= (x_1 + px_1^2) + B(x_2 + px_2^2) + B^2(x_3 + px_3^2) + B^3(x_4 + px_4^2) \\ &= x_1 + Bx_2 + B^2x_3 + B^3x_4 + p(x_1^2 + Bx_2^2 + B^2x_3^2 + B^3x_4^2). \end{aligned}$$

Điều kiện để giải mã chính xác

Các giá trị x_i phải thỏa:

$$g(x_i, p) < B, \quad i = 1, 2, 3, 4$$

Giá trị lớn nhất của $g(x, p)$ là:

$$g_{\max} = 15 + 225p$$

Do đó, điều kiện tổng quát cho (p, B) là:

$$\begin{cases} p \in \mathbb{Z}^+ \\ B \in \mathbb{Z}, \quad B > 15 + 225p \end{cases}$$

p	B_{min}
1	241
2	466
3	691
4	916
5	1141
...	...

Bảng 2.3: Mô tả các giá trị hợp lệ của B theo p

Với mỗi p cố định, B có thể là bất kỳ số nguyên nào lớn hơn $B_{\text{tối thiểu}}$.

2.3.2 Hàm giải mã

Giả sử dãy số (x_1, x_2, x_3, x_4) được mã hóa thành số nguyên y theo cơ số B :

$$y = d_1 + d_2B + d_3B^2 + d_4B^3,$$

với

$$d_i = g(x_i) = x_i + px_i^2.$$

Bước 1: Lấy các tầng d_i từ y

$$\begin{aligned} d_4 &= \left\lfloor \frac{y}{B^3} \right\rfloor, & r_3 &= y - d_4B^3, \\ d_3 &= \left\lfloor \frac{r_3}{B^2} \right\rfloor, & r_2 &= r_3 - d_3B^2, \\ d_2 &= \left\lfloor \frac{r_2}{B} \right\rfloor, & d_1 &= r_2 - d_2B. \end{aligned}$$

Bước 2: Phương trình giải cho x_i

Mỗi d_i được tính từ x_i theo công thức:

$$d_i = g(x_i) = x_i + px_i^2$$

suy ra phương trình bậc hai:

$$px_i^2 + x_i - d_i = 0$$

Bước 3: Giải phương trình bậc hai

Phương trình bậc hai tổng quát: $ax^2 + bx + c = 0$ có nghiệm:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Áp dụng với $a = p, b = 1, c = -d_i$:

$$x_i = \frac{-1 \pm \sqrt{1 - 4p(-d_i)}}{2p} = \frac{-1 \pm \sqrt{1 + 4pd_i}}{2p}$$

Bước 4: Chọn nghiệm dương nguyên

Vì x_i là số nguyên dương, chỉ chọn dấu cộng:

$$x_i = \frac{-1 + \sqrt{1 + 4pd_i}}{2p}$$

Bước 5: Quy trình giải mã đầy đủ

Bước	Biểu thức d_i	Phần dư	Giá trị x_i
1	$d_4 = \left\lfloor \frac{y}{B^3} \right\rfloor$	$r_3 = y - d_4 B^3$	$x_4 = \frac{-1 + \sqrt{1 + 4pd_4}}{2p}$
2	$d_3 = \left\lfloor \frac{r_3}{B^2} \right\rfloor$	$r_2 = r_3 - d_3 B^2$	$x_3 = \frac{-1 + \sqrt{1 + 4pd_3}}{2p}$
3	$d_2 = \left\lfloor \frac{r_2}{B} \right\rfloor$	$r_1 = r_2 - d_2 B$	$x_2 = \frac{-1 + \sqrt{1 + 4pd_2}}{2p}$
4	$d_1 = r_1 - d_2 B$	-	$x_1 = \frac{-1 + \sqrt{1 + 4pd_1}}{2p}$

Bảng 2.4: Quy trình giải mã các giá trị x_i từ y theo cơ số B và hằng số p .**2.4 CƠ SỞ GIẤU ẢNH TRONG KÊNH MÀU RGBA****2.4.1 Biểu diễn ảnh số**

Một ảnh màu có thể được biểu diễn dưới dạng hàm rời rạc:

$$I : \Omega \rightarrow \{0, 1, \dots, 255\}^3$$

trong đó:

- $\Omega = \{(x, y) \mid 0 \leq x < H, 0 \leq y < W\}$ là tập tọa độ pixel của ảnh.
- $I(x, y) = [R(x, y), G(x, y), B(x, y)]$ là vector biểu diễn cường độ ba kênh màu.

Khi mở rộng sang ảnh RGBA, mỗi pixel có thêm kênh alpha:

$$I'(x, y) = [R(x, y), G(x, y), B(x, y), A(x, y)]$$

với $A(x, y) \in [0, 255]$ là giá trị độ trong suốt (opacity) của điểm ảnh.

2.4.2 Chuyển đổi dữ liệu giấu

Giả sử ta có thông điệp M cần giấu, là chuỗi ký tự bất kỳ. Thông điệp này được mã hóa sang dạng nhị phân:

$$M_b = (b_1, b_2, b_3, \dots, b_L), \quad b_i \in \{0, 1\}$$

với $L = 8 \times \text{len}(M)$ là tổng số bit cần giấu.

Mỗi bit b_i sẽ được ánh xạ vào một pixel (x_i, y_i) của ảnh.

Chia thông điệp thành các phần dựa trên số pixel ảnh

Gọi ảnh có kích thước $H \times W$, số pixel tổng cộng là:

$$N_{\text{pixels}} = H \cdot W$$

Kích thước mỗi phần thông điệp L' được tính theo:

$$L' = N_{\text{pixels}} - 64$$

Giả sử thông điệp M có độ dài L_M . Số phần cần chia là:

$$n = \left\lceil \frac{L_M}{L'} \right\rceil$$

Phần thứ i ($i = 1, 2, \dots, n$) của thông điệp được xác định bởi:

$$M_i = M[(i-1)L' : \min(iL', L_M)]$$

Trong 64 pixel đầu tiên (tương đương 54 bit) của mỗi ảnh lưu trữ thông tin điều khiển giải thuật mã hóa, gồm:

1. **32 bit đầu tiên:** lưu trữ số lượng tổng ảnh mà giải thuật mã hóa gửi đi.
2. **32 bit tiếp theo:** lưu trữ số thứ tự phần thông điệp được cắt ra và lưu trong ảnh hiện tại.

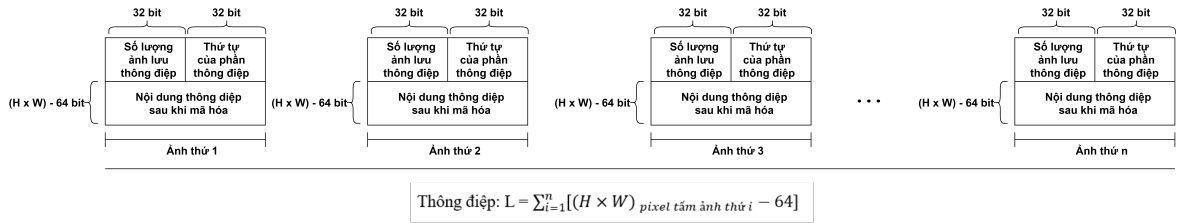
Cách thức lưu trữ tổng quát có thể biểu diễn như sau:

$$\text{Header}_{\text{image}} = \underbrace{b_1 b_2 \dots b_{32}}_{\text{số lượng ảnh}} \underbrace{b_{33} b_{34} \dots b_{64}}_{\text{số thứ tự phần thông điệp}}$$

Với mỗi ảnh thứ i trong tổng số N ảnh:

$$\begin{cases} 32 \text{ bit đầu} = \text{biểu diễn nhị phân của } N, \\ 32 \text{ bit tiếp theo} = \text{biểu diễn nhị phân của } i \end{cases}$$

Phần còn lại của ảnh được sử dụng để lưu trữ nội dung phần thông điệp tương ứng.



Hình 2.1: Tổ chức cấu trúc đầu trong ảnh

Dùng thuật toán mã hóa đối xứng DES để mã hóa 64 bit đầu của mỗi ảnh.

2.4.3 Nguyên lý nhúng dữ liệu

Thuật toán sử dụng kỹ thuật **Least Significant Bit (LSB)** để điều chỉnh bit thấp nhất của kênh alpha. Giá trị alpha được gán xấp xỉ 255 để đảm bảo không thay đổi thị giác:

$$A'(x_i, y_i) = \begin{cases} 253, & \text{nếu không chứa thông tin} \\ 254, & \text{nếu } b_i = 0, \\ 255, & \text{nếu } b_i = 1. \end{cases} \quad (2.1)$$

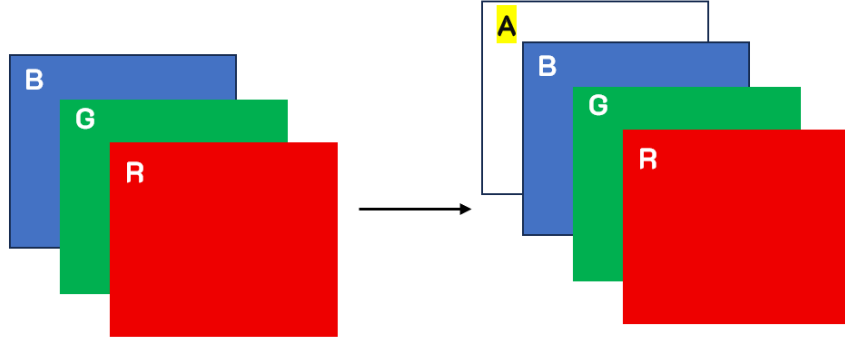
Khi đó, ảnh mã hóa được mô tả:

$$I'_{\text{encoded}}(x, y) = [R(x, y), G(x, y), B(x, y), A'(x, y)]$$

Dung lượng cực đại có thể giấu trong một ảnh là:

$$C = H \times W$$

bits dữ liệu.



Hình 2.2: Cấu trúc ảnh RGBA và kênh Alpha được sử dụng để giấu dữ liệu.

2.4.4 Giải mã dữ liệu

Tại phía thu, kênh alpha của ảnh được đọc lại và tách bit dữ liệu theo phép modulo:

$$\hat{b}_i = A'(x_i, y_i) \bmod 2 \quad (2.2)$$

Tập hợp các bit thu được:

$$\hat{M}_b = (\hat{b}_1, \hat{b}_2, \dots, \hat{b}_L)$$

được ghép lại thành chuỗi byte để tái tạo thông điệp gốc:

$$\hat{M} = \text{Decode}(\hat{M}_b)$$

Nếu ảnh không bị biến dạng:

$$\hat{M}_b = M_b \Rightarrow \hat{M} = M$$

2.4.5 Mô hình nhúng và trích xuất

Hai quá trình nhúng và trích xuất được biểu diễn bởi:

$$I' = E(I, M_b) \quad (2.3)$$

$$\hat{M}_b = D(I') \quad (2.4)$$

trong đó:

- E là hàm nhúng dữ liệu vào ảnh bằng phép biến đổi bit thấp nhất.
- D là hàm trích xuất bit từ ảnh đã mã hóa.

Tính thuận nghịch lý tưởng:

$$D(E(I, M_b)) = M_b$$

2.4.6 Độ sai khác và đánh giá chất lượng

Sai khác giữa ảnh gốc và ảnh mã hóa có thể đánh giá bằng chỉ số PSNR:

$$\text{MSE} = \frac{1}{H \times W} \sum_{x,y} (A'(x,y) - A(x,y))^2 \quad (2.5)$$

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \quad (2.6)$$

Do $|A'(x,y) - A(x,y)| \leq 1$, giá trị PSNR thường lớn hơn 50 dB, nghĩa là biến dạng hầu như không thể nhận biết bằng mắt người.

Kết luận

Phương pháp giấu tin trong kênh alpha thuộc nhóm Steganography miền không gian (Spatial Domain). Nó đảm bảo:

- Ảnh mã hóa gần như không thay đổi trực quan;
- Dữ liệu có thể khôi phục hoàn toàn nếu ảnh không bị chỉnh sửa;
- Mỗi pixel chứa được một bit dữ liệu;
- Dễ triển khai, tốc độ cao, và có thể kết hợp thêm các thuật toán mã hóa (AES, RSA) để tăng bảo mật.

2.5 CƠ SỞ THUẬT TOÁN DES - Data Encryption Standard

DES là một thuật toán mã hóa khối (block cipher) tiêu chuẩn, làm việc trên khối dữ liệu 64-bit và khóa 56-bit. DES thực hiện 16 vòng Feistel, với các bước chính gồm: hoán vị ban đầu (IP), vòng Feistel, và hoán vị ngược (IP^{-1}).

1. Mã hóa (Encryption)

1. **Input:** Khối plaintext M (64-bit), khóa K (56-bit).
2. **Bước 1: Hoán vị ban đầu (Initial Permutation, IP)**

$$M' = IP(M)$$

3. **Bước 2: Chia khối thành hai nửa 32-bit**

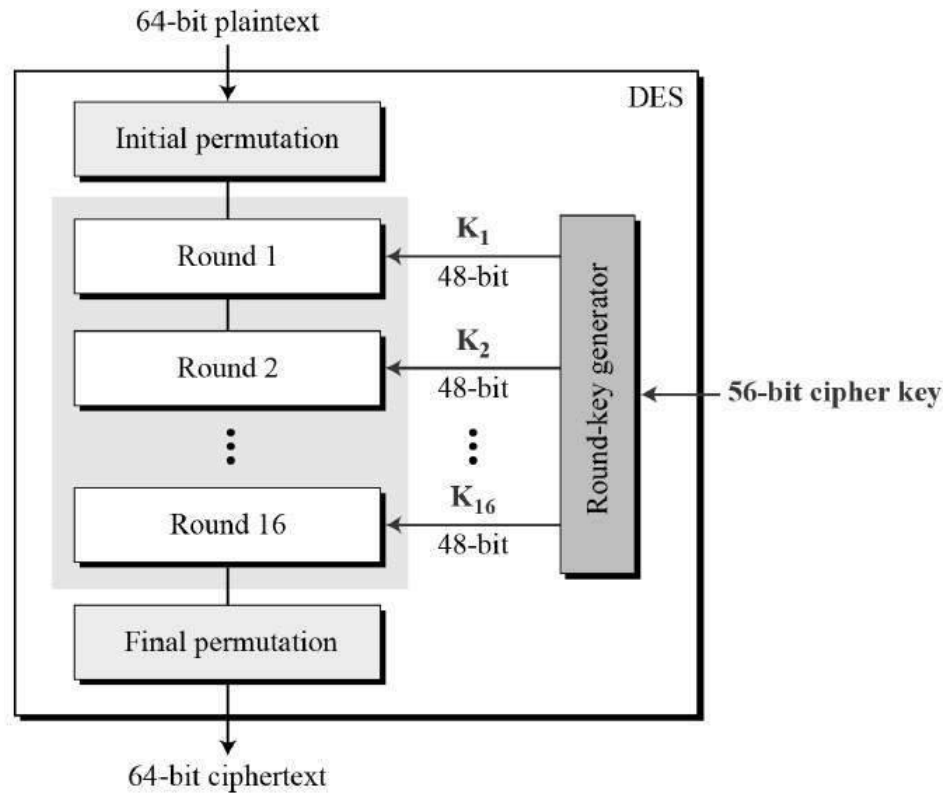
$$L_0 = \text{nửa trái của } M', \quad R_0 = \text{nửa phải của } M'$$

4. **Bước 3: 16 vòng Feistel**

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i), \quad i = 1, 2, \dots, 16$$

Trong đó:

- K_i là subkey 48-bit cho vòng i (từ K qua Permuted Choice 1 và Left Shift)



Hình 2.3: Cấu trúc của giải thuật DES

- F là hàm Feistel:

$$F(R, K_i) = P(S(E(R) \oplus K_i))$$

với:

- E : Expansion 32-bit $\rightarrow$ 48-bit
- S : 8 hộp S-box 6-bit $\rightarrow$ 4-bit
- P : Permutation 32-bit

5. Bước 4: Ghép ngược nửa cuối và nửa trái

$$M'' = R_{16} || L_{16}$$

6. Bước 5: Hoán vị ngược (Inverse IP)

$$C = IP^{-1}(M'')$$

7. Output: Ciphertext C (64-bit)

2. Giải mã (Decryption)

Giải mã DES sử dụng cùng các bước như mã hóa, chỉ khác là sử dụng các subkey theo thứ tự ngược.

1. Input: Ciphertext C (64-bit), khóa K (56-bit)

2. Thực hiện hoán vị ban đầu:

$$C' = IP(C)$$

3. Chia C' thành hai nửa:

$$L_0 = \text{nửa trái của } C', \quad R_0 = \text{nửa phải của } C'$$

4. 16 vòng Feistel với subkey đảo ngược:

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_{17-i}), \quad i = 1, 2, \dots, 16$$

5. Ghép ngược và hoán vị ngược:

$$M = IP^{-1}(R_{16} \| L_{16})$$

6. **Output:** Plaintext M (64-bit)

3. Ghi chú

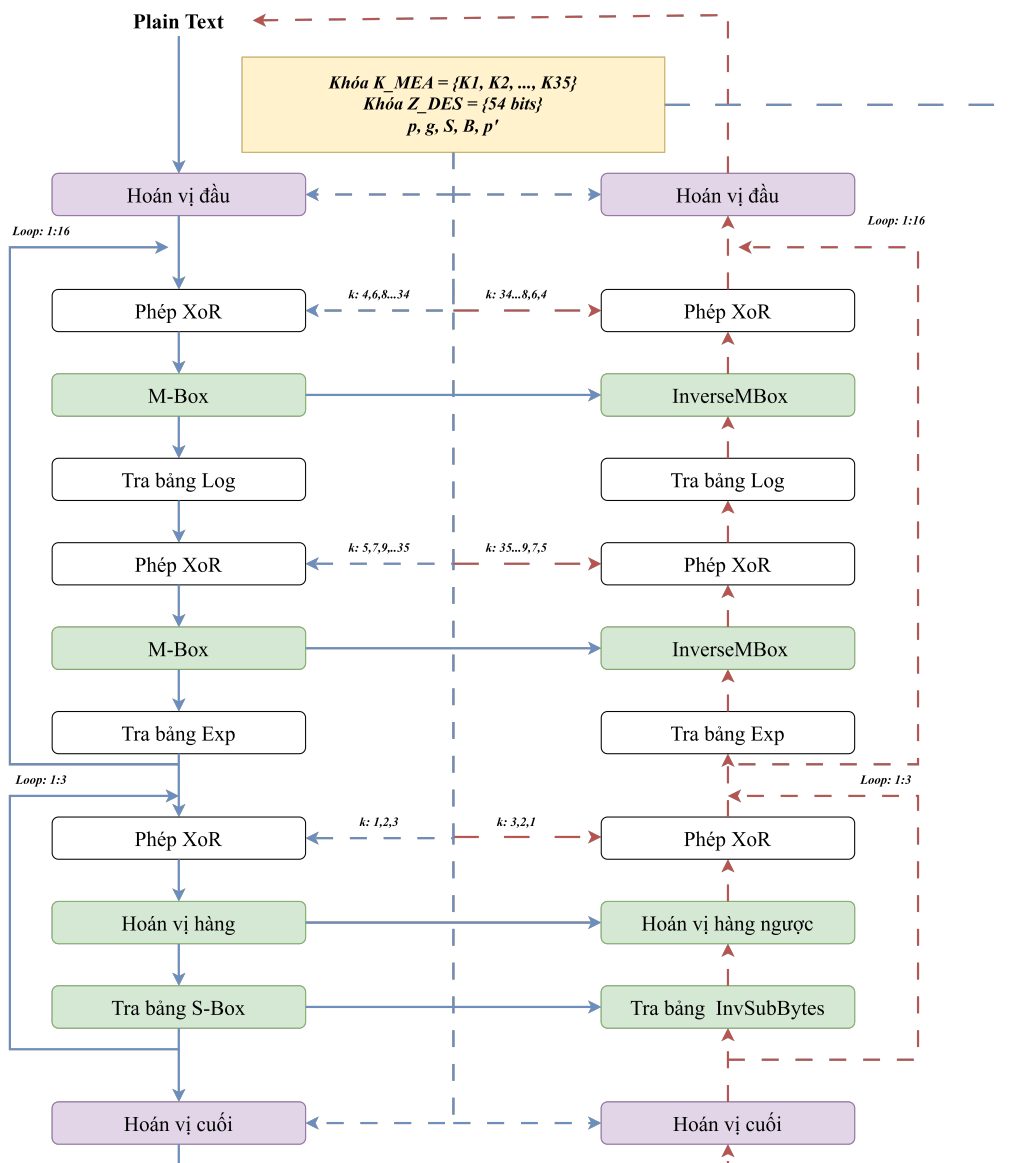
- DES là thuật toán khối, đối xứng, cùng một khóa K để mã hóa và giải mã.
- 64-bit khối dữ liệu bao gồm 56-bit thực và 8-bit parity.
- Sử dụng 16 vòng Feistel giúp DES an toàn trước nhiều loại tấn công cơ bản.

2.6 Kết luận

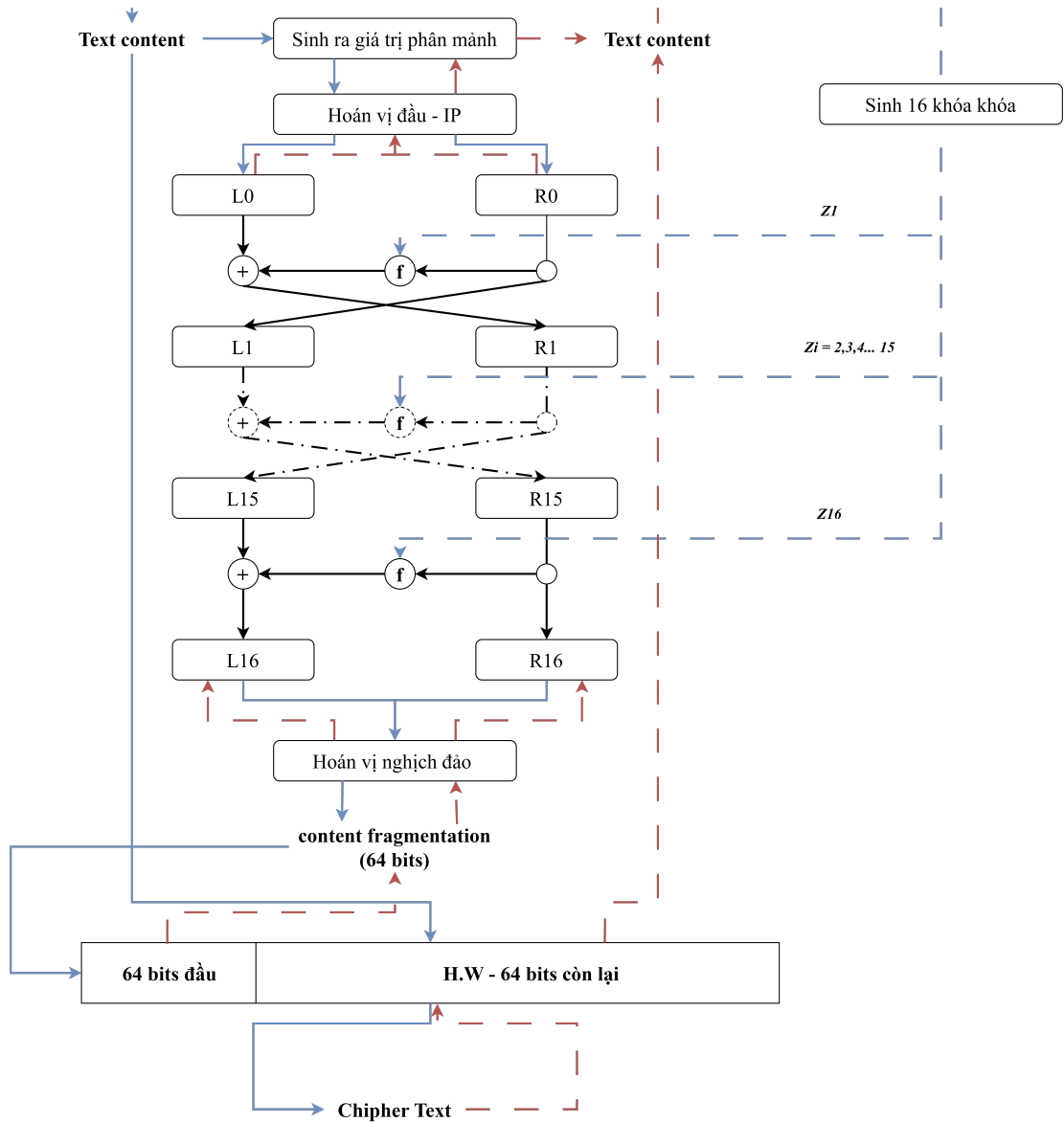
- MEA gốc là một thuật toán mã hóa đối xứng mạnh mẽ nhờ cấu trúc ma trận tri-modular và M-Box.
- Phiên bản cải tiến của nhóm em không chỉ tăng mức độ hỗn loạn và khuếch tán, mà còn mở rộng ứng dụng sang lĩnh vực giấu tin ảnh, giúp tăng cường bảo mật, tính toán học và giá trị thực tiễn.

Chương 3

QUY TRÌNH THỰC HIỆN THUẬT TOÁN MEA (Modular Encryption Algorithm) CẢI TIẾN



Hình 3.1: Sơ đồ giải thuật Modular Encryption Algorithm cải tiến phần 1



Hình 3.2: Sơ đồ giải thuật Modular Encryption Algorithm cải tiến phần 2

3.1 Sinh Khóa

Quy trình sinh khóa gồm 5 bước chính:

1. **Hoán vị đầu:** Sử dụng một số nguyên tố lớn p , với g là phần tử sinh của nhóm nhân $\mathbb{F}_p^\times$. Chọn ma trận khả nghịch $S \in GL(3, \mathbb{F}_p)$ để thực hiện quá trình hoán vị trên phép toán ma trận.
2. **Trộn và khuếch đại dữ liệu:** Dữ liệu gốc được trộn và khuếch đại độ nhiễu thông qua 35 ma trận tam thức (các ma trận 3×3 hoặc theo cấu trúc tam thức phù hợp).
3. **Ánh xạ phi tuyến:** Dữ liệu sau khi trộn được ánh xạ trên phép toán phi tuyến bậc 2 bằng hai giá trị p và B , trong đó B là số nguyên dương phụ thuộc vào p . (công thức cụ thể phụ thuộc vào thuật toán phi tuyến sử dụng).
4. **Mã hóa bằng DES:** Giá trị phân mảnh 64-bit đầu tiên của mỗi ảnh chứa dữ liệu mã hóa được mã hóa bằng thuật toán DES, với khóa Z là dãy 64-bit sinh ngẫu nhiên.

nhiên:

$$C = \text{DES}_Z(P)$$

trong đó P là dữ liệu 64-bit cần mã hóa, C là kết quả mã hóa.

5. **Giấu dữ liệu và chuẩn hóa:** Dữ liệu đã mã hóa được giấu vào tập ảnh và tiến hành chuẩn hóa để giảm độ nhiễu, đảm bảo ảnh đầu ra chứa dữ liệu nhưng vẫn giữ chất lượng hình ảnh tốt.

Khóa tổng hợp Key_{total} của toàn bộ quy trình sinh dữ liệu gồm các thành phần:

$$Key_{\text{total}} = \left(p, g, S, \{M_i\}_{i=1}^{35}, p', B, Z \right)$$

Bảng 3.1: Cấu trúc khóa trong 5 bước sinh dữ liệu

Bước	Khóa / Giá trị	Mô tả
1	$p, g, S \in GL(3, \mathbb{F}_p)$	Số nguyên tố p , phần tử sinh g của nhóm nhân, ma trận khả nghịch S dùng hoán vị dữ liệu.
2	$\{K_i\}_{i=1}^{35}$	35 ma trận tam thức dùng trộn và khuếch đại độ nhiễu dữ liệu.
3	B, p'	Ánh xạ phi tuyến bậc 2; B là số nguyên dương phụ thuộc vào p' .
4	Z (DES 64-bit)	Khóa DES dùng mã hóa 64-bit đầu mỗi ảnh:
5	-	Không tạo khóa mới; giấu dữ liệu đã mã hóa vào ảnh và chuẩn hóa.

3.2 Thuật toán mã hóa

Đầu vào (Input): Chuỗi bản rõ (*Plaintext*) được chia thành các khối gồm 9 ký tự. Mỗi ký tự 8-bit được biểu diễn bằng 2 giá trị hexadecimal 4-bit. Do đó, mỗi ô trong ma trận 3×3 sẽ chứa 2 giá trị hex 4-bit, đảm bảo mỗi khối 9 ký tự tương ứng với $9 \times 8 = 72$ bit dữ liệu.

$$B^{(1)} = \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix}, \quad b_{ij} = (h_1, h_2), \quad h_1, h_2 \in [0, F]_{\text{hex}}$$

Hóa vị đầu

Hàm mã hóa

$$f(X) = S[g^{x_{ij} \bmod(p)}]S^{-1}$$

Trong đó:

- $[g^{x_{ij}} \bmod(p)]$ là ma trận cấp số nhân, với g là phần tử sinh của nhóm nhân $\mathbb{F}_p^\times$ trên từng giá trị của $B^{(1)}$.
- S^{-1} là ma trận nghịch đảo của S .

Kết quả:

$$A^{(2)} = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix}, \quad a_{ij} = (h_1, h_2), \quad h_1, h_2 \in [0, F]_{\text{hex}}$$

Pha I (Phase-I): Vòng lặp 16 lần

For loop: $i = 1$ đến 16 thực hiện các bước sau:

1. **Phép XOR với khóa:**

$$A^{(3)} = A^{(2)} \oplus K_i$$

trong đó K_i là ma trận khóa tam thức (*Tri-Modular matrix*) với $4 \leq i \leq 34$.

2. **Phép nén (Compression Permutation)** qua hộp M-Box:

$$A^{(4)} = M\text{-Box}(A^{(3)})$$

3. **Bảng logarit (Log Table):**

$$A^{(5)} = \log(A^{(4)})$$

4. **Phép XOR với khóa mới:**

$$A^{(6)} = A^{(5)} \oplus K_i$$

với $5 \leq i \leq 35$.

5. **Phép nén lần 2:**

$$A^{(7)} = M\text{-Box}(A^{(6)})$$

6. **Bảng mũ (Exponential Table):**

$$A^{(8)} = \exp(A^{(7)})$$

Pha II (Phase-II): Vòng lặp 3 lần

For loop: $i = 1$ đến 3 thực hiện các bước sau:

1. **Phép XOR với khóa tam thức:**

$$A^{(9)} = A^{(8)} \oplus K_i, \quad 1 \leq k \leq 3$$

2. **Hoán vị hàng (Row Interchange):**

$$R_i \leftarrow R_{(i+1) \bmod 3}, \quad 0 \leq i \leq 2$$

tức là hàng thứ i được thay thế bằng hàng kế tiếp (theo chu kỳ modulo 3).

3. Phép thay thế byte (Sub-Bytes):

$$A^{(11)} = S\text{-Box}(A^{(10)})$$

Sau khi dữ liệu gốc (*plain text*) được trộn và khuếch đại qua 2 pha, toàn bộ ký tự được chuyển về dạng hexadecimal, ví dụ:

$$[\text{CipherText}]_1 = 01 \ 23 \ 45 \ 67 \ 89 \ AB \ CD \ EF \ \dots$$

Trong đó:

- Mỗi ký tự gốc 8-bit được biểu diễn bằng 2 ký tự hex.
- Chuỗi được nối tiếp theo thứ tự xử lý, phản ánh trạng thái dữ liệu sau 2 pha khuếch đại.
- Dấu ‘...’ biểu thị phần còn lại của chuỗi.

Chú ý: Đây là dạng dữ liệu chuẩn để tiếp tục các bước hóa mã cuối. Kết quả của bước này là

$$[\text{CipherText}]_1.$$

Hoán vị cuối

Chuỗi $[\text{CipherText}]_1$ được chia thành các khối 4 ký tự và sắp xếp thành ma trận 2×2 :

$$T = \begin{bmatrix} t_{00} & t_{01} \\ t_{10} & t_{11} \end{bmatrix} \quad t_{ij} = (h_1, h_2), \quad h_1, h_2 \in [0, F]_{\text{hex}}$$

Tính giá trị y mới

$$y = (t_{00} + pt_{00}^2) + B(t_{01} + pt_{01}^2) + B^2(t_{10} + pt_{10}^2) + B^3(t_{11} + pt_{11}^2)$$

Giá trị cao nhất của y là 999,999,999,999 (nếu thiếu theo giá trị 0 vào trước để đủ 12 số). Tách y theo từng cụm 2 số liên tiếp nhau kết quả của bước này là chuỗi nhị phân được ánh xạ đặt là:

$$[\text{CipherText}]_2.$$

Sinh ra giá trị phân mảnh

$$\text{Số lượng ảnh} = \frac{\text{Chiều dài } ciphertext_1}{512 \times 512 - 64}$$

1. Mã hóa giá trị phân mảnh

1. **Input:** Khối plaintext M (64-bit), khóa K (56-bit).

2. **Bước 1: Hoán vị ban đầu (Initial Permutation, IP)**

$$M' = IP(M)$$

3. **Bước 2: Chia khối thành hai nửa 32-bit**

$$L_0 = \text{nửa trái của } M', \quad R_0 = \text{nửa phải của } M'$$

4. Bước 3: 16 vòng Feistel

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i), \quad i = 1, 2, \dots, 16$$

Trong đó:

- K_i là subkey 48-bit cho vòng i (từ K qua Permuted Choice 1 và Left Shift)
- F là hàm Feistel:

$$F(R, K_i) = P(S(E(R) \oplus K_i))$$

với:

- E : Expansion 32-bit $\rightarrow$ 48-bit
- S : 8 hộp S-box 6-bit $\rightarrow$ 4-bit
- P : Permutation 32-bit

5. Bước 4: Ghép ngược nửa cuối và nửa trái

$$M'' = R_{16} \| L_{16}$$

6. Bước 5: Hoán vị ngược (Inverse IP)

$$C = IP^{-1}(M'')$$

7. Output: Ciphertext C (64-bit)

Giấu vào ảnh

Mỗi ảnh sinh ra lớp ảo thứ 4 là A' sau 3 tầng RGB lưu trữ: $C_{DES}[CipherText]_2$

$$A(b_i, b_j) = \begin{cases} 253, & \text{nếu } b_i \text{ là không chẵn thông tin gốc,} \\ 254, & \text{nếu } b_i = 0, \\ 255, & \text{nếu } b_i = 1. \end{cases}$$

Khi đó, ảnh mã hóa được mô tả:

$$I'_{\text{encoded}}(x, y) = [R(x, y), G(x, y), B(x, y), A'(x, y)]$$

Đầu ra (Output): Kết quả cuối cùng của giải thuật mã hóa MEA mở rộng (cipher text) là tập hình ảnh chứa thông tin được mã hóa ẩn sau tầng ẩn

3.3 Thuật toán giải mã

Đầu vào (Input): Tập hình ảnh chứa các thông điệp mã hóa

Tách thông điệp từ ảnh

Ảnh mã hóa được mô tả:

$$I'_{\text{encoded}}(x, y) = [R(x, y), G(x, y), B(x, y), A'(x, y)]$$

Mỗi ảnh tách tầng ẩn A' ra, khi đó thông điệp gốc phục hồi A dạng:

$$A(b_i, b_j) = \begin{cases} 253, & \text{nếu } b_i \text{ là không chứa thông tin gốc,} \\ 254, & \text{nếu } b_i = 0, \\ 255, & \text{nếu } b_i = 1. \end{cases} \quad (3.1)$$

Giải mã giá trị phân mảnh

1. **Input:** Ciphertext C (64-bit), khóa K (56-bit)

2. Thực hiện hoán vị ban đầu:

$$C' = IP(C)$$

3. Chia C' thành hai nửa:

$$L_0 = \text{nửa trái của } C', \quad R_0 = \text{nửa phải của } C'$$

4. 16 vòng Feistel với subkey đảo ngược:

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_{17-i}), \quad i = 1, 2, \dots, 16$$

5. Ghép ngược và hoán vị ngược:

$$M = IP^{-1}(R_{16} \| L_{16})$$

6. **Output:** Plaintext M (64-bit)

Sau khi giải mã giá trị phân mảnh xong, tiến hành ghép các thông điệp bắt đầu từ bit thứ 65 trở đi của mỗi ảnh theo thứ tự để thông điệp hoàn chỉnh đã mã hóa là $[\text{CipherText}]_1$.

Hoán vị ngược của hoán vị cuối

Chuỗi $[\text{CipherText}]_1$ được chia thành các khối 4 chuỗi 8 bit và chuyển về **2 giá trị thập lục phân (hexadecimal)** và ghép lại thành 1 dãy số gồm 12 chữ số:

Giải mã về 4 giá trị gốc:

Bước	Biểu thức d_i	Phần dư	Giá trị x_i
1	$d_4 = \left\lfloor \frac{y}{B^3} \right\rfloor$	$r_3 = y - d_4 B^3$	$t_{11} = \frac{-1 + \sqrt{1 + 4pd_4}}{2p}$
2	$d_3 = \left\lfloor \frac{r_3}{B^2} \right\rfloor$	$r_2 = r_3 - d_3 B^2$	$t_{10} = \frac{-1 + \sqrt{1 + 4pd_3}}{2p}$
3	$d_2 = \left\lfloor \frac{r_2}{B} \right\rfloor$	$r_1 = r_2 - d_2 B$	$t_{01} = \frac{-1 + \sqrt{1 + 4pd_2}}{2p}$
4	$d_1 = r_1 - d_2 B$	-	$t_{00} = \frac{-1 + \sqrt{1 + 4pd_1}}{2p}$

Bảng 3.2: Quy trình giải mã các giá trị t_{ij} từ y theo cơ số B và hằng số p .

Giá trị gốc:

$$T = \begin{bmatrix} t_{00} & t_{01} \\ t_{10} & t_{11} \end{bmatrix} \quad t_{ij} = (h_1, h_2), \quad h_1, h_2 \in [0, F]_{\text{hex}}$$

Hoán vị ngược của hoán vị đầu

Chuỗi bản rõ (*Plaintext*) được chia thành các khối gồm 9 ký tự. Mỗi ký tự 8-bit được biểu diễn bằng 2 giá trị hexadecimal 4-bit. Do đó, mỗi ô trong ma trận 3×3 sẽ chứa 2 giá trị hex 4-bit, đảm bảo mỗi khối 9 ký tự tương ứng với $9 \times 8 = 72$ bit dữ liệu.

$$M^{(1)} = \begin{bmatrix} m_{00} & m_{01} & m_{02} \\ m_{10} & m_{11} & m_{12} \\ m_{20} & m_{21} & m_{22} \end{bmatrix}, \quad m_{ij} = (h_1, h_2), \quad h_1, h_2 \in [0, F]_{\text{hex}}$$

Hàm giải mã:

$$g(M^{(1)}) = \left(\log_g \left((S^{-1} M^{(1)} S)_{ij} \right) \bmod (p-1) \right)_{ij}.$$

Trong đó:

- Phép $\log_g()$ được thực hiện trên trường số nguyên mô-đun $\mathbb{Z}_{p-1}$ (hay còn gọi là trường dư theo $p-1$ nếu $p-1$ là số nguyên tố), hoặc chính xác hơn là vành $\mathbb{Z}_{p-1}$ nếu $p-1$ không nguyên tố.
- S^{-1} là ma trận nghịch đảo của S .

Kết quả:

$$M^{(2)} = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix}, \quad a_{ij} = (h_1, h_2), \quad h_1, h_2 \in [0, F]_{\text{hex}}$$

Pha I (Phase-I): Vòng lặp 3 lần

For loop: $i = 1$ đến 3 thực hiện các bước sau:

1. Chuyển đổi thập phân sang hệ 16 (Decimal $\rightarrow$ Hex):

$$M^{(3)} = \text{Dec2Hex}(M^{(2)})$$

2. Phép *Inverse SubBytes*:

$$M^{(4)} = \text{InvSubBytes}(M^{(3)})$$

3. Hoán vị hàng (Row Interchange): với quy tắc tuần hoàn

$$R_{(i+1) \bmod 3} \leftarrow R_i, \quad 0 \leq i \leq 2$$

thu được $M^{(5)}$.

4. XOR với khóa tam thức:

$$M^{(6)} = M^{(5)} \oplus K_i$$

trong đó K_i là các ma trận khóa tam thức ứng với $1 \leq k \leq 3$.

Pha II (Phase-II): Vòng lặp 16 lần

For loop: $j = 1$ đến 16 thực hiện các bước sau:

1. Tra bảng logarithm:

$$M^{(7)} = \log(M^{(6)})$$

2. Phép nén/khôi phục mở rộng (Expansion permutation) qua *Inverse M-Box*:

$$M^{(8)} = \text{InverseMBox_Expand}(M^{(7)})$$

3. XOR với khóa tam thức:

$$M^{(9)} = M^{(8)} \oplus K_j \quad (\text{với } 5 \leq k \leq 35 \text{ theo quy định khóa})$$

4. Tra bảng Exponential:

$$M^{(10)} = \exp(M^{(9)})$$

5. Lại phép mở rộng qua *Inverse M-Box*:

$$M^{(11)} = \text{InverseMBox_Expand}(M^{(10)})$$

6. XOR với khóa tam thức:

$$M^{(12)} = M^{(11)} \oplus K_j \quad (\text{với } 4 \leq k \leq 34 \text{ theo quy định khóa})$$

7. Gán $M^{(6)} \leftarrow M^{(12)}$ cho vòng kế tiếp (nếu có).

Bước 12: Chuyển từ hệ Hex về hệ thập phân

$$M^{(13)} = \text{Hex2Dec}(M^{(12)})$$

Bước 13: Chuyển sang ký tự ASCII

$$M^{(14)} = \text{ASCII}(M^{(13)})$$

Đầu ra (Output): Khối bản rõ (Plaintext block) cuối cùng gồm 9 ký tự, tương ứng 72 bit:

$$\text{Plaintext} = M^{(14)}$$

Chương 4

KẾT QUẢ THỰC NGHIỆM

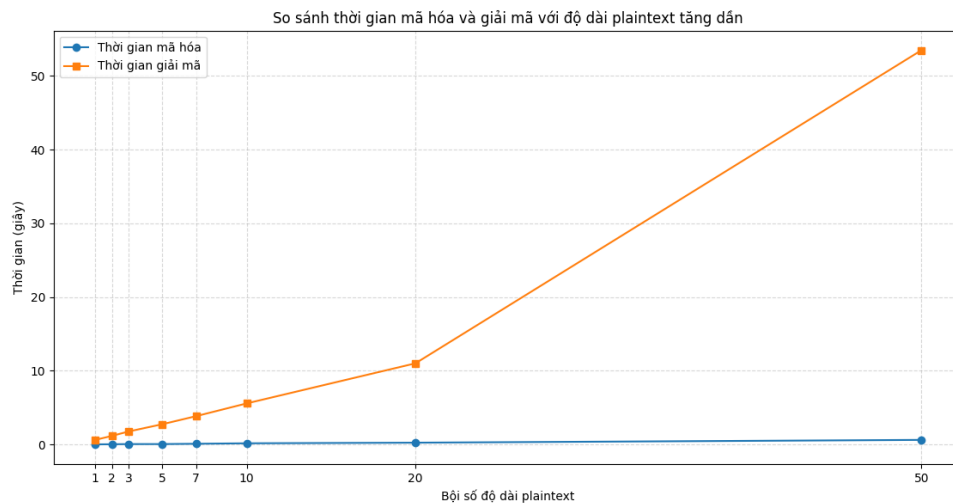
Để đánh giá hiệu năng và độ an toàn của giải thuật **MEA (Modified Encryption Algorithm)**, quá trình mã hóa và giải mã được thực hiện trong cùng một môi trường máy tính nhằm đảm bảo tính nhất quán về thời gian xử lý và tài nguyên phần cứng.

Bảng 4.1: Cấu hình máy tính và môi trường thực nghiệm

Thành phần	Thông số
Hệ điều hành	Windows 11 Home Single Language
Bộ xử lý (CPU)	Intel Core i5-1135G7 @ 2.40GHz (4 nhân, 8 luồng)
Bộ nhớ RAM	8 GB DDR4
Ổ cứng	SSD 512 GB
Ngôn ngữ lập trình	Python 3.10.11
Thư viện hỗ trợ	NumPy, typing, matplotlib, math, Cryptodome, Image, ...
IDE sử dụng	Visual Studio Code
Ảnh thử nghiệm	RGB, tùy ý, định dạng PNG, JPG, ...

Để đánh giá mức độ cải thiện của thuật toán, nhóm tiến hành đo và so sánh các tiêu chí thực thi giữa phiên bản **MEA gốc** và **MEA cải tiến**. Các phép đo được thực hiện trên cùng một cấu hình phần cứng (Bảng 4.1), cùng kích thước ảnh 512×512 và cùng bộ khóa đầu vào. **Plaintext:**

This is a group I information security and safety report in English.



Hình 4.1: Đánh giá mô hình MEA

Bảng 4.2: So sánh thời gian thực thi giữa MEA và MEA cải tiến

Thuật toán	Thời gian sinh khóa (s)	Thời gian mã hóa (s)	Thời gian giải mã (s)
MEA gốc	1.340	0.16	1.076
MEA cải tiến	1.405	0.350	1.458

Bảng 4.2 trình bày so sánh thời gian thực thi giữa thuật toán MEA gốc và MEA cải tiến. Có thể thấy rằng thời gian sinh khóa của MEA cải tiến chỉ tăng nhẹ so với MEA gốc (1.405 giây so với 1.340 giây). Tuy nhiên, thời gian mã hóa và giải mã tăng đáng kể (0.350 giây và 1.458 giây so với 0.16 giây và 1.076 giây), phản ánh chi phí tính toán cao hơn do các bước cải tiến nhằm nâng cao bảo mật. Nhìn chung, MEA cải tiến có mức độ bảo mật cao hơn nhưng đánh đổi bằng thời gian thực thi lâu hơn.

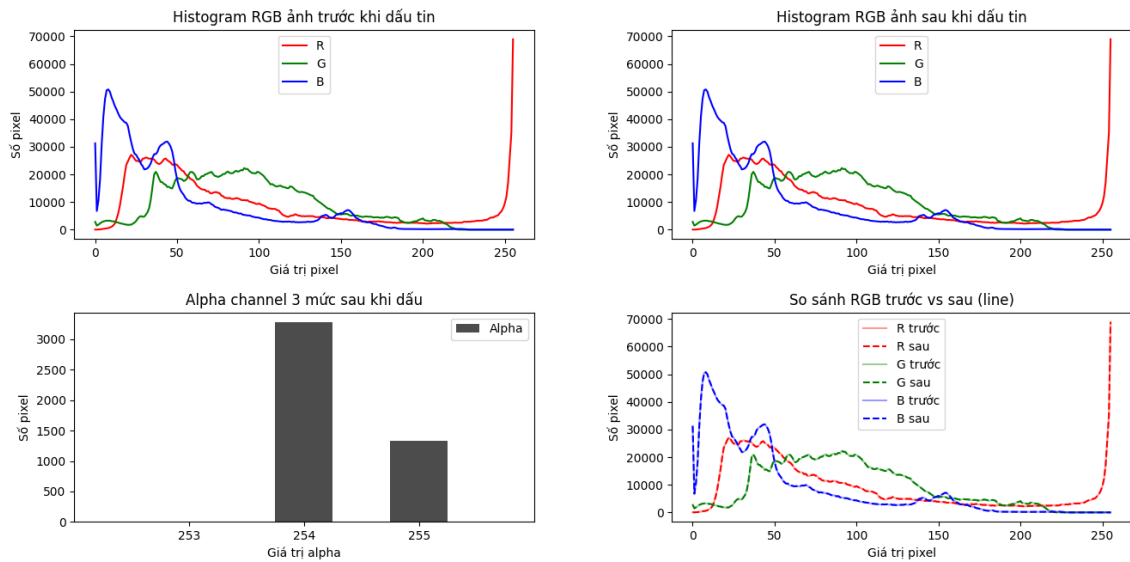


(a) Ảnh gốc



(b) Ảnh sau khi giấu tin

Hình 4.2: So sánh giữa ảnh gốc và ảnh sau khi giấu tin



Hình 4.3: So sánh histogram màu ảnh trước và sau khi giấu tin

Bảng 4.3: So sánh chất lượng ảnh giữa ảnh gốc và ảnh sau khi giấu tin

Chỉ số	Giá trị	Nhận xét
MSE	0	Hoàn toàn giống ảnh gốc
PSNR (dB)	148.131	Rất cao, chất lượng gần như không thay đổi
Entropy Original (bits)	7.369	Mức độ ngẫu nhiên trên kênh RGB không thay đổi do dữ liệu giấu tin được nhúng vào kênh alpha, không tác động đến các kênh màu gốc. Việc này giúp duy trì độ ngẫu nhiên và cấu trúc màu gốc, từ đó làm khó hơn quá trình phân tích và phá giải của hacker, đồng thời đảm bảo mã hóa và giải mã diễn ra chính xác mà không làm biến dạng ảnh.
Entropy Encoded (bits)	7.369	

Bảng 4.3 trình bày các chỉ số chất lượng giữa ảnh gốc và ảnh sau khi giấu tin. Giá trị MSE bằng 0 và PSNR rất cao (148.131 dB) cho thấy ảnh giấu tin gần như không thay đổi về mặt trực quan. Entropy trên kênh RGB của cả ảnh gốc và ảnh giấu tin đều giữ nguyên (7.369 bits), do dữ liệu giấu tin được nhúng vào kênh alpha, không tác động đến các kênh màu gốc. Việc này giúp duy trì độ ngẫu nhiên và cấu trúc màu gốc, từ đó làm khó hơn quá trình phân tích và phá giải của hacker, đồng thời đảm bảo mã hóa và giải mã diễn ra chính xác mà không làm biến dạng ảnh.

Avalanche Effect là một đặc tính quan trọng trong mật mã học, thể hiện rằng khi thay đổi một bit trong *plaintext* hoặc trong *key*, ít nhất 50% các bit trong *ciphertext* sẽ thay đổi. Mục đích đánh giá hiệu ứng này là để kiểm tra mức độ nhạy cảm của thuật toán mã hóa đối với các thay đổi nhỏ, từ đó đánh giá khả năng chống lại các tấn công dựa trên sự phụ thuộc giữa *plaintext*, *key* và *ciphertext*. Thuật toán càng đạt Avalanche gần 50% thì càng an toàn. Công thức tính Avalanche được cho bởi:

$$\text{Avalanche} = \frac{\text{Số bit thay đổi trong ciphertext}}{\text{Tổng số bit của ciphertext}} \times 100\%.$$

Bảng 4.4: Minh họa Avalanche Effect với các số bit thay đổi khác nhau

Số bit thay đổi plan text	Số bit thay đổi cipher text	Avalanche
1	54	5.11%
10	106	10.04%
50	313	29.64%
200	403	38.16%

Nhận xét: Bảng trên cho thấy thuật toán có hiệu ứng Avalanche, tức thay đổi nhỏ ở plaintext dẫn đến thay đổi đáng kể ở ciphertext. Mức Avalanche chưa tối ưu, dao động từ 5% đến 38%, nhưng xu hướng chung là càng thay đổi nhiều bit plaintext, số bit khác ở ciphertext càng tăng.

Chương 5

TỔNG KẾT VÀ ĐỊNH HƯỚNG PHÁT TRIỂN

5.1 BÀI HỌC TIẾP THU

Trong quá trình nghiên cứu và triển khai đề tài, nhóm chúng em đã học hỏi được nhiều kiến thức quý giá từ bài báo “Modular Encryption Algorithm for Secure Data Transmission” của P. S. R. Chandra, G. Venkateswara Rao và G. V. Swamy. Bài báo cung cấp cái nhìn sâu sắc về cách thiết kế thuật toán mã hóa dựa trên ma trận modular, cách kết hợp các phép thay thế và hoán vị để tăng độ phức tạp, cũng như các tiêu chí đánh giá bảo mật như avalanche effect. Những kiến thức này không chỉ giúp nhóm nắm vững lý thuyết, mà còn hỗ trợ trong việc phân tích, cải tiến thuật toán MEA và triển khai các ứng dụng giấu tin ảnh một cách khoa học và hiệu quả.

Bên cạnh đó, nhóm em đã được tiếp cận với phương pháp đọc và nghiên cứu bài báo khoa học một cách bài bản hơn. Nhờ đó nhóm đã biết cách xác định mục tiêu nghiên cứu, đọc tóm tắt, phân tích phương pháp mà còn học được cách đánh giá kết quả và rút ra hướng phát triển cho đề tài. Những kỹ năng này giúp việc tiếp cận các tài liệu khoa học trở nên dễ dàng và là nền tảng quan trọng cho quá trình học tập và nghiên cứu trong tương lai.

5.2 ƯU ĐIỂM

Thuật toán MEA cải tiến mà nhóm nghiên cứu đề xuất đã thể hiện nhiều ưu điểm nổi bật. Trước hết, việc cải thiện các vòng mã hóa và cấu trúc M-Box giúp gia tăng đáng kể độ phức tạp và tính ngẫu nhiên của bản mã, nhờ đó nâng cao khả năng chống lại các phương pháp tấn công phân tích tuyến tính cũng như các tấn công sai biệt. Đây là một bước tiến quan trọng nhằm tăng tính bảo mật so với MEA gốc, đồng thời đảm bảo rằng các thông tin nhạy cảm không dễ bị giải mã hoặc dự đoán bởi các hacker.

Bên cạnh đó, thuật toán còn tích hợp khả năng giấu tin trong ảnh RGBA bằng kỹ thuật LSB, kết hợp với các thuật toán mã hóa truyền thống như DES và IDEA. Việc này không chỉ nâng cao mức độ bảo mật của dữ liệu nhúng mà còn mở rộng tính ứng dụng của thuật toán sang lĩnh vực bảo vệ dữ liệu đa phương tiện. Nhóm nghiên cứu nhận thấy rằng sự kết hợp này cho phép bảo vệ đồng thời cả dữ liệu gốc và thông tin nhúng, tạo ra một hệ thống an toàn hơn cho người dùng trong môi trường mạng hiện nay.

Ngoài ra, quá trình phát triển thuật toán đã giúp nhóm hiểu rõ quy trình xây dựng

một hệ thống mã hóa hoàn chỉnh, từ sinh khóa, mã hóa, giấu tin đến giải mã và phục hồi dữ liệu. Việc nắm vững cơ chế hoạt động của các khối toán học nâng cao như ánh xạ tuyến tính trên $GL(3, F_p)$, phép liên hợp ma trận, logarit rời rạc và ánh xạ phi tuyến bậc hai cũng góp phần nâng cao chất lượng nghiên cứu, đồng thời mở ra cơ hội ứng dụng thuật toán trong các hệ thống bảo mật dữ liệu phức tạp hơn trong tương lai.

5.3 HẠN CHẾ

Mặc dù đạt được nhiều ưu điểm, thuật toán MEA cải tiến vẫn còn tồn tại một số hạn chế đáng chú ý. Trước hết, sự gia tăng độ phức tạp của thuật toán dẫn đến chi phí tính toán cao hơn, yêu cầu phần cứng mạnh hơn và thời gian mã hóa, giải mã dài hơn so với phiên bản gốc. Điều này có thể ảnh hưởng đến khả năng triển khai trong các hệ thống thời gian thực hoặc trên các thiết bị có tài nguyên hạn chế.

Thứ hai, hiện tại thuật toán chưa được thử nghiệm trên các bộ dữ liệu thực tế lớn, vì vậy hiệu năng tổng thể, độ ổn định và khả năng mở rộng của hệ thống vẫn chưa được kiểm chứng đầy đủ. Việc này đặt ra thách thức trong việc áp dụng vào các ứng dụng thực tế, đặc biệt là trong môi trường mạng với lượng dữ liệu khổng lồ.

Ngoài ra, một hạn chế quan trọng là thuật toán hiện chưa đảm bảo bảo mật tuyệt đối cho thông tin gốc khi dữ liệu bị hacker can thiệp hoặc sửa đổi trên đường truyền. Trong trường hợp thông tin bị thay đổi, hệ thống có thể không phát hiện kịp thời, dẫn đến nguy cơ mất mát hoặc thay đổi dữ liệu quan trọng. Đây là điểm yếu cần được khắc phục để thuật toán có thể ứng dụng an toàn trong môi trường có rủi ro cao về bảo mật.

5.4 HƯỚNG PHÁT TRIỂN

Trên cơ sở kết quả hiện tại, nhóm đề xuất nhiều hướng phát triển để hoàn thiện thuật toán MEA cải tiến. Một trong những hướng quan trọng là tối ưu hóa hiệu năng xử lý, bao gồm giảm thời gian mã hóa và giải mã, tận dụng tốt hơn tài nguyên phần cứng, đồng thời nâng cao khả năng mở rộng cho các tập dữ liệu lớn. Việc này sẽ giúp thuật toán phù hợp hơn với các ứng dụng thực tế, kể cả trên thiết bị giới hạn tài nguyên.

Thứ hai, thử nghiệm thuật toán trên các bộ dữ liệu thực tế đa dạng là cần thiết để đánh giá toàn diện về độ ổn định, khả năng triển khai và độ tin cậy của hệ thống. Đây cũng là bước quan trọng để xác định những cải tiến cần thiết nhằm nâng cao hiệu quả bảo mật và độ chính xác trong việc giấu tin và giải mã dữ liệu.

Một hướng đi tiềm năng khác là ứng dụng trí tuệ nhân tạo (AI) vào việc sinh khóa động. Việc này sẽ giúp hệ thống tăng tính thích ứng, tự động hóa quá trình sinh khóa và nâng cao mức độ bảo mật, đồng thời giảm thiểu rủi ro khi dữ liệu bị tấn công. Ngoài ra, nhóm cũng có kế hoạch mở rộng phương pháp giấu tin sang các định dạng khác như video, ảnh nhiều lớp, hoặc các định dạng media phức tạp hơn, từ đó tăng khả năng ứng dụng trong thực tế.

Cuối cùng, nghiên cứu về cơ chế bảo vệ dữ liệu gốc khi bị can thiệp trên đường truyền sẽ giúp hệ thống trở nên toàn diện hơn. Việc bổ sung các thuật toán kiểm tra, xác thực hoặc phát hiện thay đổi dữ liệu sẽ tăng cường độ tin cậy và đảm bảo an toàn cho người sử dụng, đặc biệt trong môi trường mạng công cộng hoặc các hệ thống nhạy cảm về bảo mật.

Tài liệu tham khảo

- [1] P. S. R. Chandra, G. Venkateswara Rao, và G. V. Swamy, “Modular Encryption Algorithm for Secure Data Transmission,” *International Journal of Scientific Research in Network Security Communication*
- [2] A. M. Abed và đồng tác giả, “Secure Image Steganography Combined with DES Encryption,” *WC (World Conference)*, 2015.
- [3] S. Gangurde, K. Tiwari, “Image Steganography using LSB and encrypted message with AES, RSA, DES, 3DES and Blowfish”, 2020.
- [4] M. Khan, M. Sajjad, I. Mehmood, S. Rho và S. W. Baik, “A novel magic LSB substitution method (M-LSB-SM) using multi-level encryption and achromatic component of an image”, 2015
- [5] G. Paul, I. Mukherjee, “Image Sterilization to Prevent LSB-based Steganographic Transmission”, 2010.
- [6] A. Chadha, N. Satam, “An Efficient Method for Image and Audio Steganography using Least Significant Bit (LSB) Substitution” , 2013.
- [7] S. Haimour, M. R. AL-Mousa, R. R. Marie, “Using Chaotic Stream Cipher to Enhance Data Hiding in Digital Images”, 2021.
- [8] D. Das, A. Durafe, V. Patidar, “An Efficient Light-weight LSB steganography with Deep learning Steganalysis” , 2022.
- [9] L. H. Triều, H. V. Canh, “Xây dựng thuật toán dấu tin mật trong ảnh số,” *JSTIC – Journal of Science and Technology on Information Communications*, 2019.
- [10] H. T. Hương Thơm, “Kỹ thuật phát hiện mù cho ảnh giấu tin trên miền LSB,” luận văn, HPU, 2013.
- [11] Đại học Duy Tân, “Bài 02 Giấu tin LSB – Một số kỹ thuật giấu tin trong ảnh,” *Khoa Công nghệ Thông tin*, trang web trường, 2018.
- [12] “Giới thiệu về kỹ thuật giấu thông tin,” *AnToanThongTin.vn*, 2016.
- [13] F. Özbudak, “Uniqueness of F_q -quadratic perfect nonlinear maps from F_q^3 to F_q^2 ,” *Finite Fields and Their Applications*, vol. 27, pp. 87–98, 2014.
- [14] D. Bartoli, “Investigating rational perfect nonlinear functions”, *Journal of Number Theory*, 2023.
- [15] R. Chen, “Classification of quadratic forms over finite fields with maximal and minimal Artin–Schreier curves”, Nov. 2024.

- [16] J. Park, “Discrete Dynamics over Finite Fields: Dynamics of special nonlinear maps over finite fields,” luận án tiến sĩ, Clemson University, 2009.
- [17] Y. Edel, A. Pott, “A new almost perfect nonlinear function which is not quadratic,” Designs, Codes and Cryptography, 2018.
- [18] G. Lido, “A provably quasi-polynomial algorithm for the discrete logarithm problem in finite fields of small characteristic”, 2022.
- [19] J. M. Pollard, “Pollard’s kangaroo algorithm for discrete logarithms,” Mathematics of Computation, 1978.